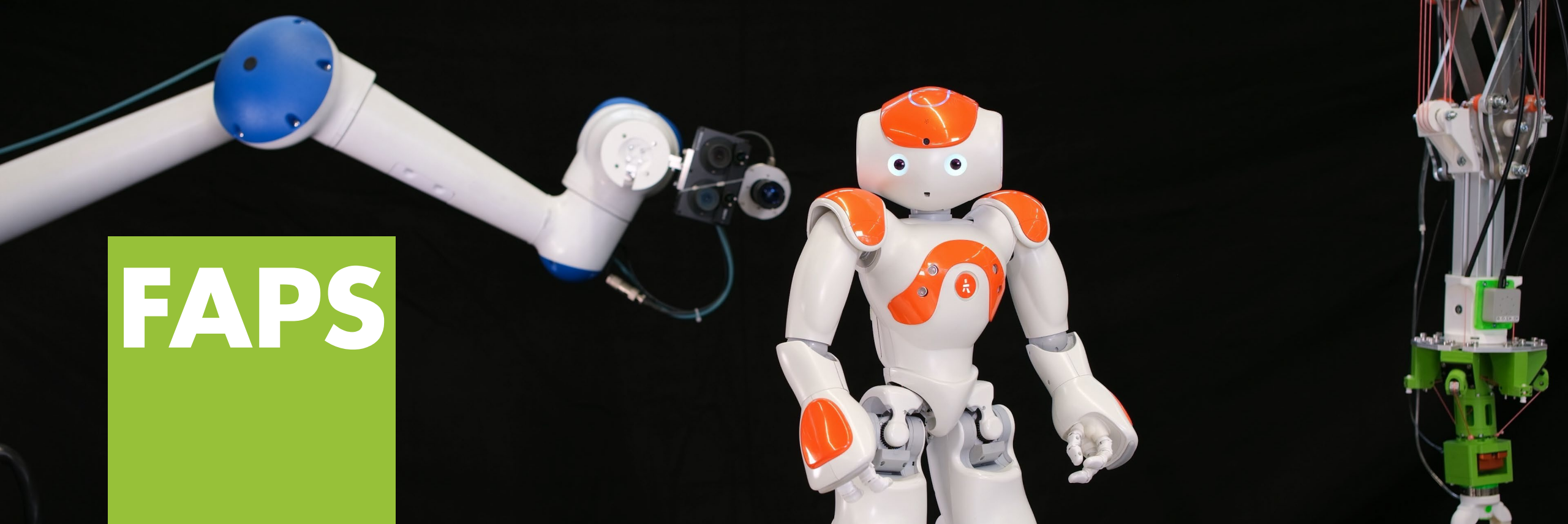




Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

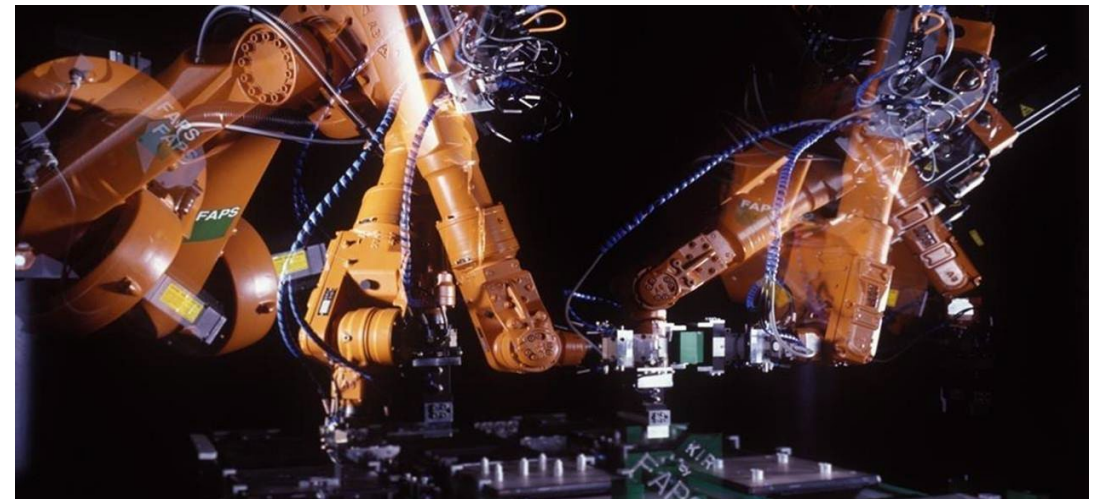
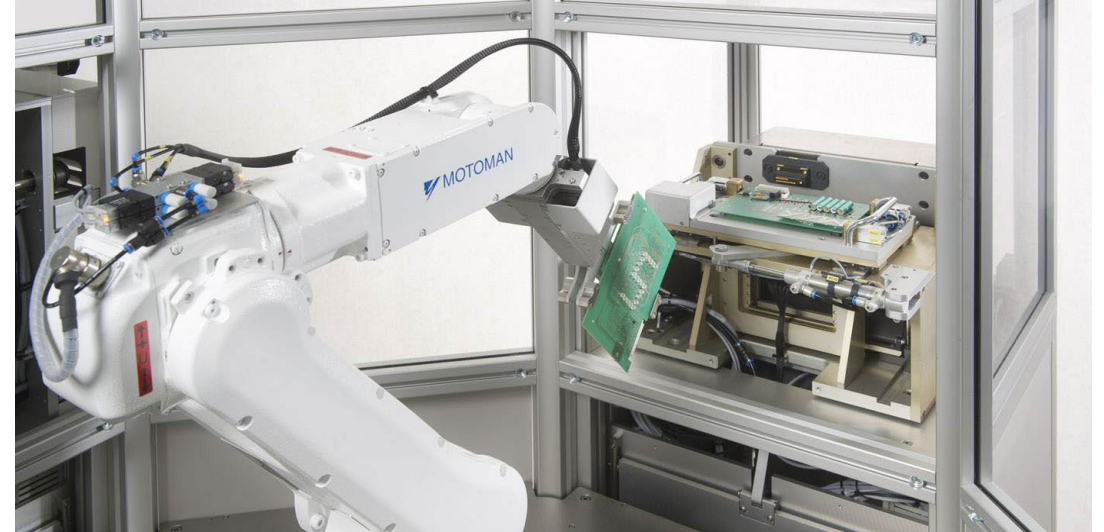
Grundlagen der Robotik (GdR)

VE08 – Simulation von Robotersystemen

Im Fach Grundlagen der Robotik werden die fachspezifischen Grundlagen zur Konzeption, Implementierung und Realisierung von Robotersystemen vermittelt.

VE	Thema
----	-------

- | | |
|----|--|
| 1 | Einführung und Grundlagen der Industrierobotik & Neuartige Konzepte und Kinematiken in der Robotik |
| 2 | Steuerung - Sensorik - Aktorik |
| 3 | Roboterkinematik |
| 4 | Roboterdynamik |
| 5 | Bahnplanung und Regelung |
| 6 | Grundlagen des Robot Operating System (ROS) |
| 7 | ROS – Erweiterte Konzepte und Tools |
| 9 | Simulation von Robotersystemen |
| 10 | Rechnersehen |
| 11 | Maschinelles Lernen |
| 12 | Seilrobotik |
| 12 | Service-, Pflege- und Medizinrobotik |



In der Vorlesungseinheit „Simulation von Robotersystemen“ wird das grundlegende Vorgehen zum Aufbau und zur Durchführung einer Robotersimulation erläutert.

Leistungsfähige Simulationswerkzeuge bilden die Grundlage zur effizienten Entwicklung und Evaluierung fortschrittlicher Robotersysteme und -anwendungen.

Nach dem Besuch dieser Vorlesungseinheit sollten Sie in der Lage sein:

- Die Bedeutung von Simulationen für die Entwicklung von Robotersystemen zu beurteilen,
- für die Robotik relevante Simulationsarten einzuordnen,
- die Potentiale und Limitierungen von Simulationen zu nennen,
- die bei der Robotersimulation notwendigen Schritte und Einflussfaktoren zu kennen,
- das grundlegende Vorgehen zur Erstellung einer Simulation zu erläutern,
- die Bedeutung von Verifikation und Validierung zu kennen.

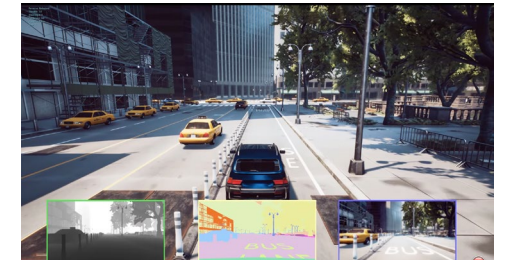


Image: Microsoft

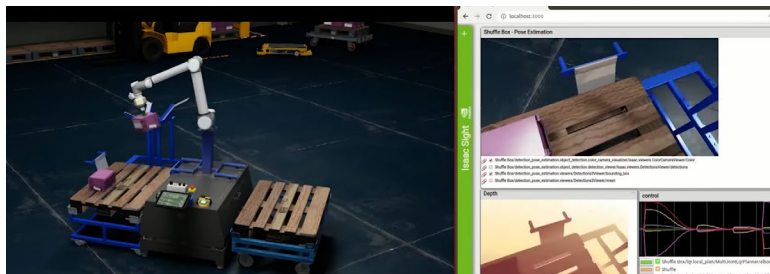


Image: NVIDIA

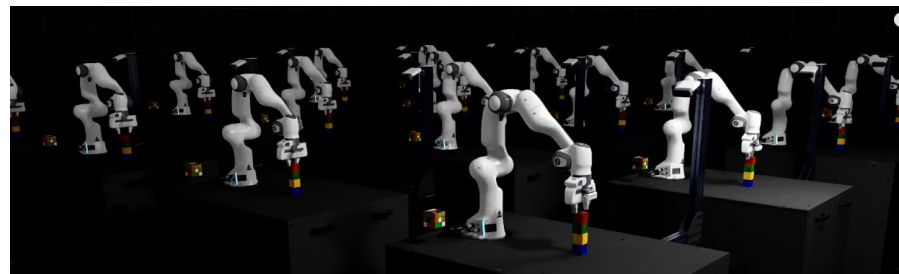


Image: NVIDIA

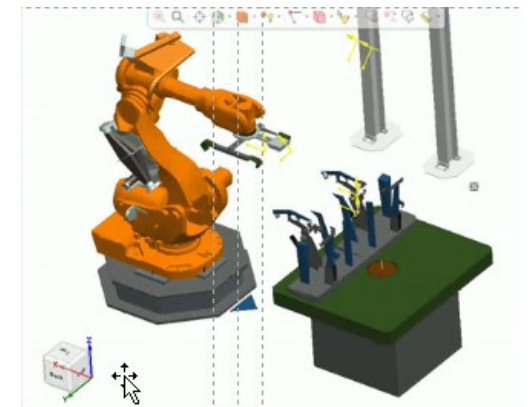


Image: Siemens

In der Vorlesung wird das Vorgehen zur Erstellung einer Robotersimulation erläutert

- Grundlegendes Vorgehen zur Erstellung von Simulationsmodellen
- Starr- und Mehrkörpersimulation
- Kollisionserkennung
- Sensor- und Aktorsimulation
- Steuerungssimulation
- Verifikation und Validierung von Simulationen
- Anwendungsbeispiel und Ausblick auf die Übung



Image: NVIDIA

Durch Simulationen können Fehler im Entwicklungsprozess frühzeitig erkannt werden, Kosten reduziert und Funktionen ohne reale Hardwareaufbauten validiert werden.



Simulationen ermöglichen unter anderem die



Frühzeitige Erkennung von
Implementierungsfehlern



Funktionsvalidierung ohne
reale Testaufbauten

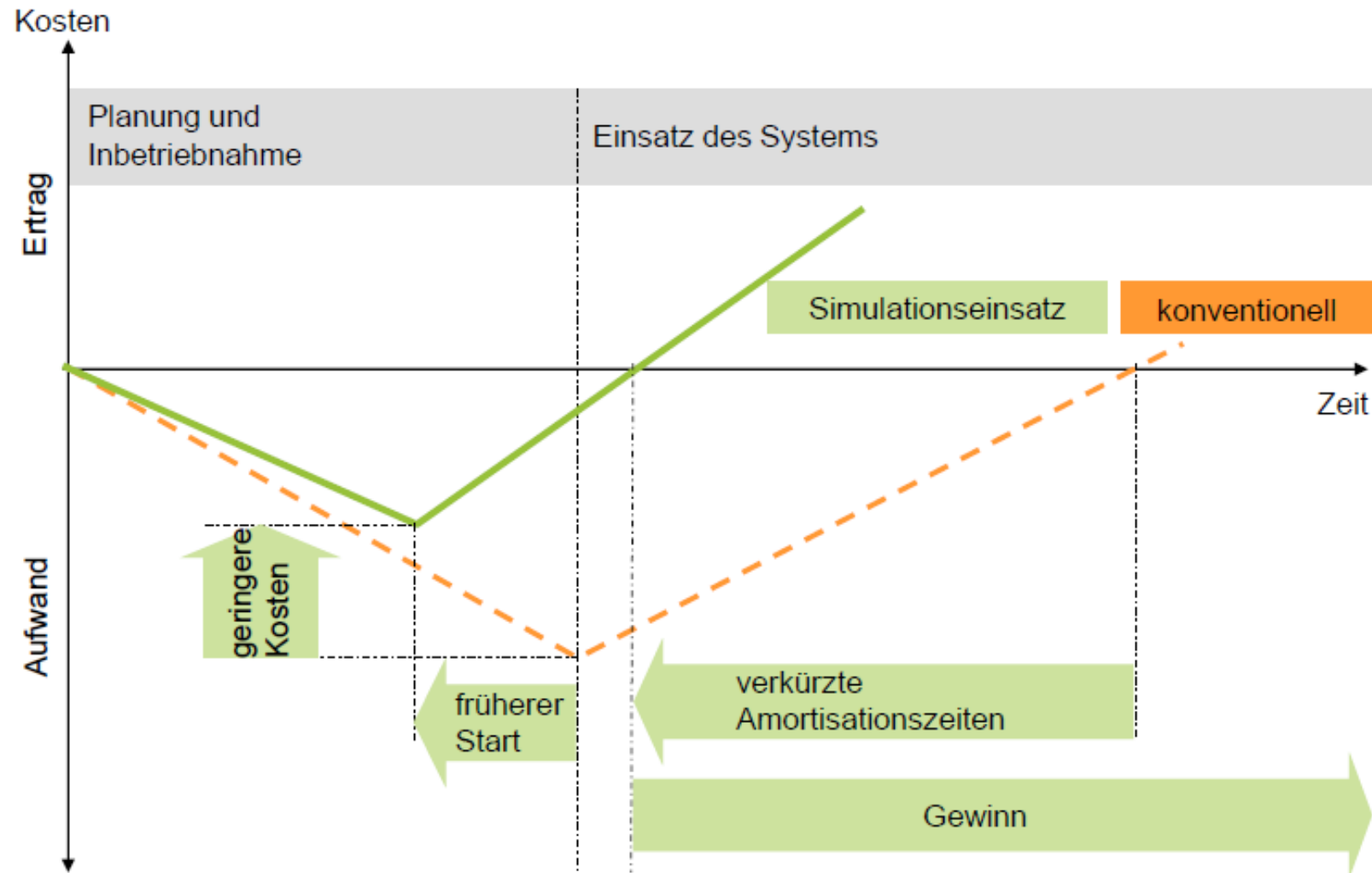


Vermeidung von Schäden
am realen Roboter



Einsparung von Zeit- und
Kosten bei der Entwicklung

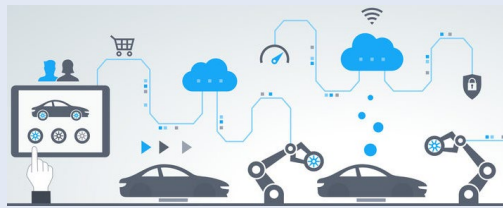
Bei der Planung von Robotersystemen führt der begleitende Einsatz von Simulationen zu erheblichen Kosteneinsparungen.



Der Einsatz von Simulationswerkzeugen ermöglicht eine beschleunigte und kostenreduzierten Entwicklung neuer Softwarekomponenten und Robotersysteme.

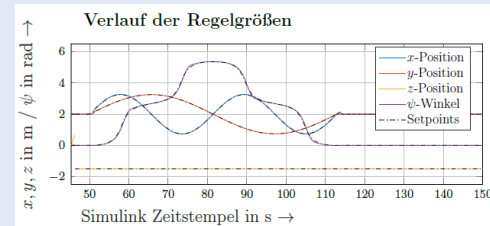
In der Robotik eingesetzte Simulationsarten (Auswahl)

Aufbausimulation

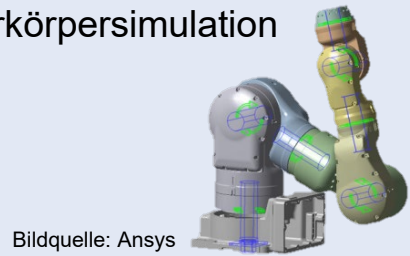


Bildquelle: entrotec.de

Regelkreissimulation

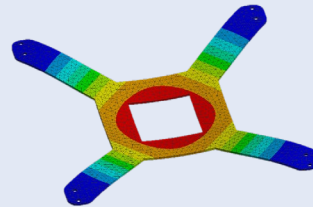


Mehrkörpersimulation

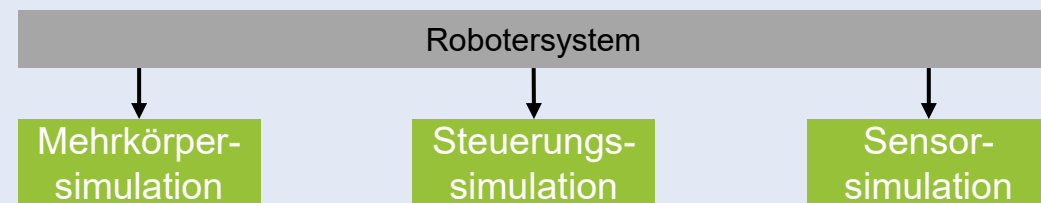


Bildquelle: Ansys

Finite-Element-Simulation



Co-simulation



Zwei oder mehr Simulationsprogramme simulieren zeitgleich unterschiedliche Teilsysteme eines komplexen Systems.

Vorteile von Simulationen

- Möglichkeit zur Evaluierung von Softwaremodulen und Algorithmen ohne Funktionsmuster und Testaufbauten
- Absicherung der korrekten Funktionsweise ohne Gefahr einer Beschädigung des Robotersystems
- Zeit- und Kostenersparnis bei der Entwicklung von Robotersystemen

Grenzen und Limitierungen von Simulationen

- Um präzise Ergebnisse zu erzielen, muss das Simulationsmodell sehr genau mit dem realen Modell übereinstimmen.
- Unerkannte falsche Simulationsergebnisse können zu fehlerhaften Entwicklungen führen.
- Eine Simulation stellt stets eine Vereinfachung der Realität dar und muss auch als solche behandelt werden.

Zur Simulation von realen Systemen werden diese in einem Modell nachgebildet, um Rückschlüsse auf die Wirklichkeit ableiten zu können.

Begriffsdefinitionen (nach VDI 3633)

Definition Simulation

Nachbildung eines Systems mit seinen dynamischen Prozessen in einem experimentierfähigen Modell, um zu Erkenntnissen zu gelangen, die auf die Wirklichkeit übertragbar sind.

Definition System

Von ihrer Umwelt abgegrenzte Menge von Elementen, die miteinander in Beziehung stehen.

Definition Modell

Vereinfachte Nachbildung eines geplanten oder existierenden Systems mit seinen Prozessen in einem anderen begrifflichen oder gegenständlichen System.

Definition Modellierung, Modellbildung

Umsetzen eines geplanten oder existierenden Systems in ein Modell.

Beispiel:

Simulation des Federungsverhaltens zur Bestimmung von Erschütterungen bei Fahrt auf unebener Fahrbahn

Betrachtetes System

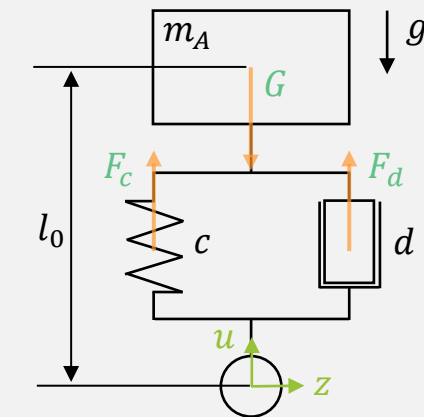


Image: Mercedes



Image: Mercedes

Modellbildung



Vereinfachtes Simulationsmodell

Model based on: Rill, G., Scheaffer, T.: „Grundlagen und Methodik der Mehrkörpersimulation“, Springer, 2017; ISBN 978-3-658-16008-1

Zur Durchführung einer Simulation muss zunächst das zu betrachtende System definiert und in Form eines mathematischen Modells abgebildet werden.

Schritte einer Simulation (nach [1])

Modellkonzept und Modellstruktur entwickeln

Entwicklung des Simulationsmodelle

Simulation durchführen

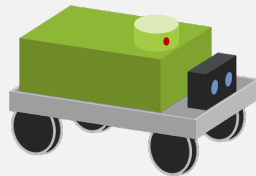
Überprüfen des Simulationsmodells

Dokumentation

[1] Bossel, H. Systeme, Dynamik, Simulation: Modellbildung, Analyse und Simulation komplexer Systeme (2004)



Bildquelle: Grenzebach

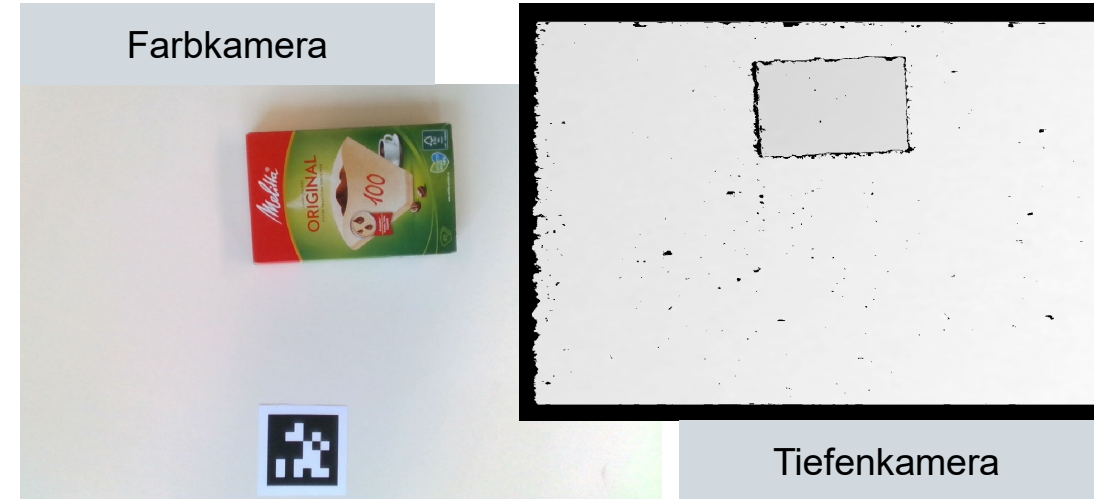
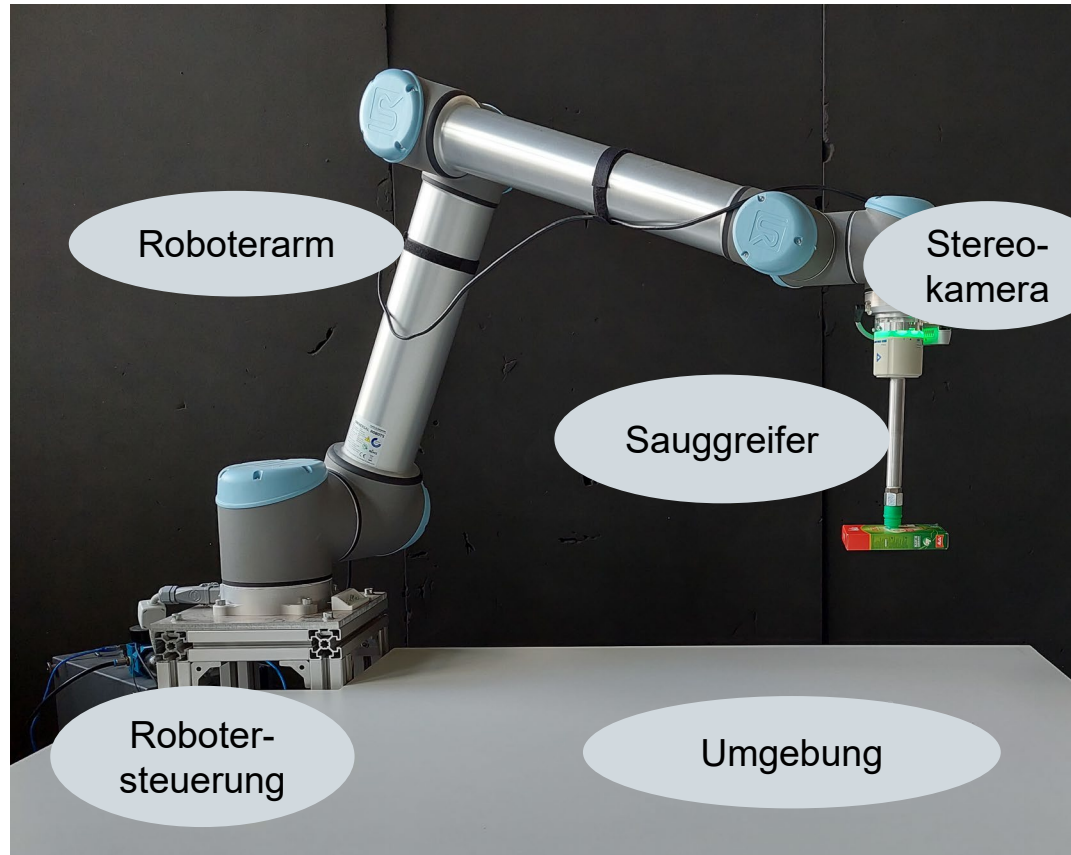


- Definition des zu betrachtenden Systems / Festlegung der Modellgrenzen
- Auswahl der benötigten Simulationsverfahren
- Definition der Eingangs- und Ausgangsgrößen
- Unterteilung des Modells in einzelne Teilmodelle, die zueinander über bekannte Schnittstellen in Beziehung stehen
- Definition des mathematischen Zusammenhangs zwischen den Ein- und Ausgangsgrößen sowie den einzelnen Teilmodellen
- Parametrisierung des resultierenden mathematischen Modells
- Festlegen von Anfangswerten
- Numerische Lösung der Modellgleichungen
- Prüfung der Simulation auf Plausibilität
- Quantitative Analyse in möglichen Gleichgewichtslagen
- Vergleich der Simulation mit Messdaten realer Systeme
- Schriftliche Dokumentation des Simulationsergebnisse

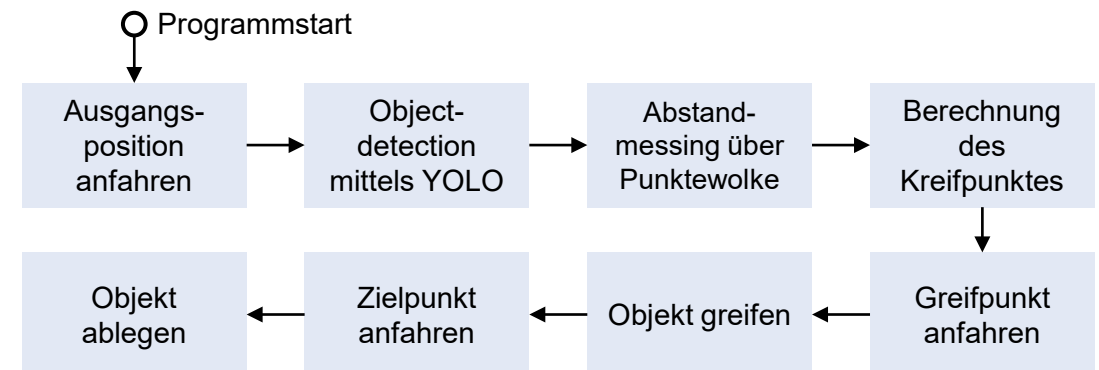
Im weiteren Verlauf der Vorlesung wird das Vorgehen zur Erstellung eines Systemmodells und dessen Simulation an einem konkreten Beispiel erläutert.



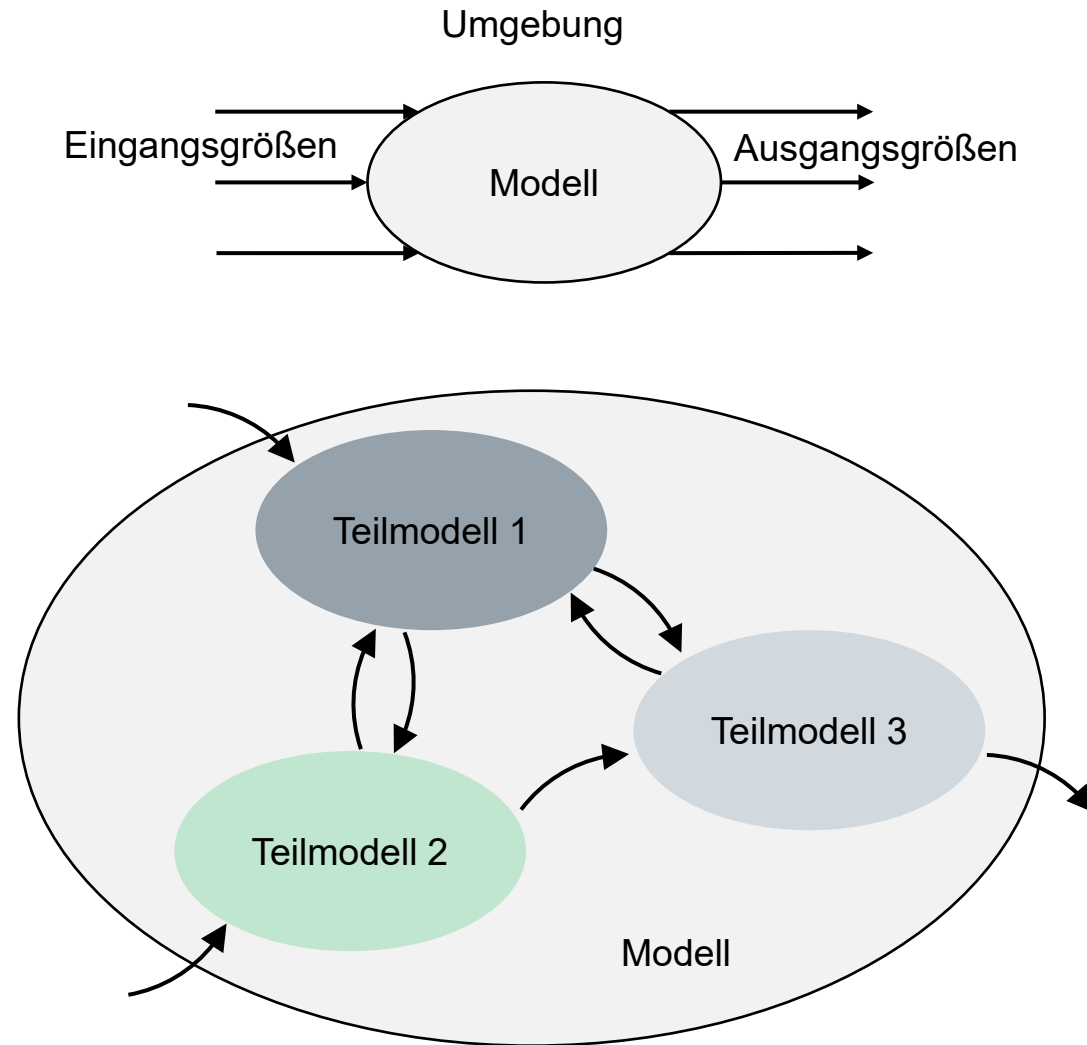
Im weiteren Verlauf der Vorlesung wird das Vorgehen zur Erstellung eines Systemmodells und dessen Simulation an einem konkreten Beispiel erläutert.



Steuerungslogik



Die Entwicklung und Modellierung der Modellstruktur und der Wechselwirkungen zwischen Ein- und Ausgangsgrößen ist ein zentrales Element bei der Erstellung einer Simulation.

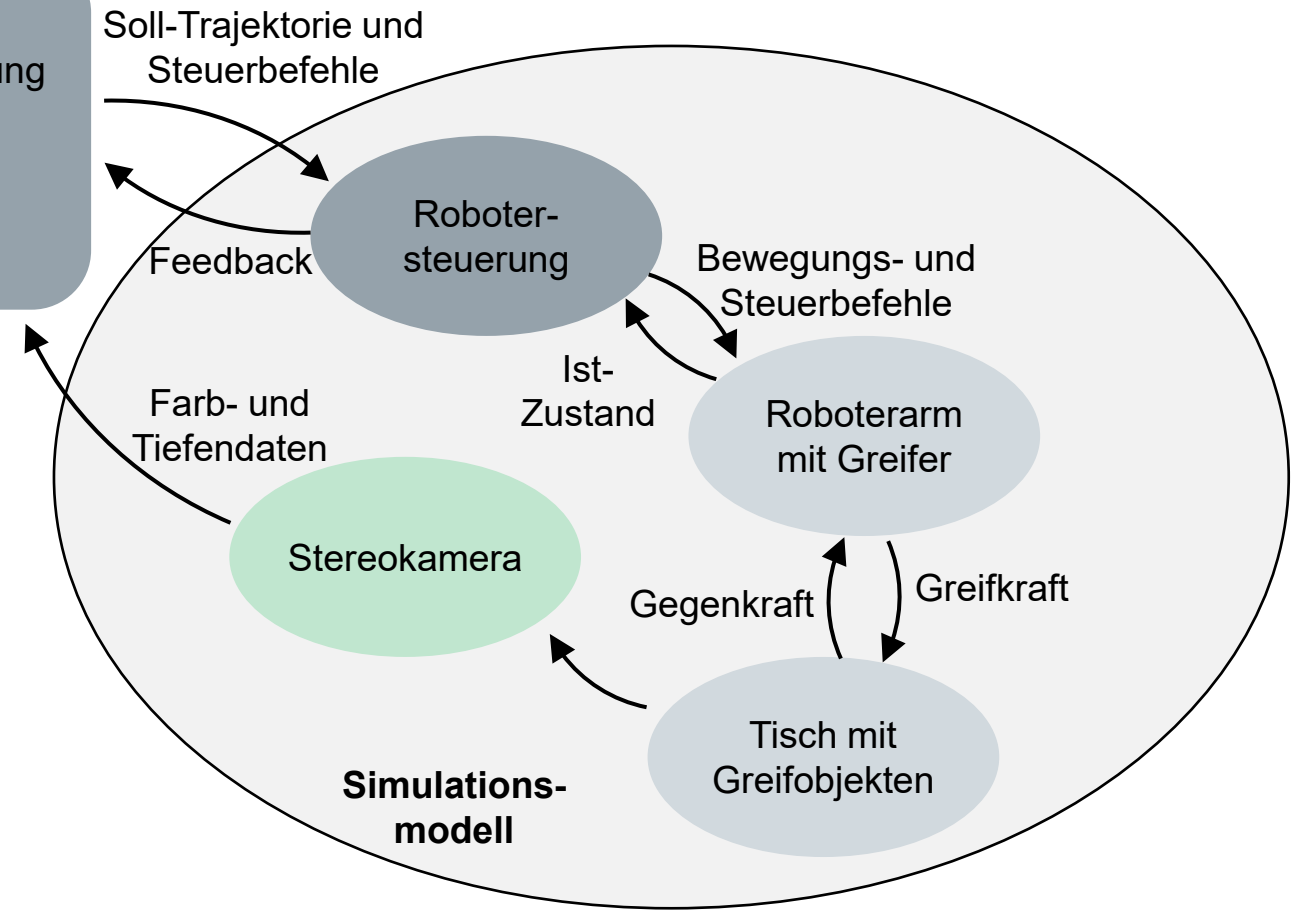
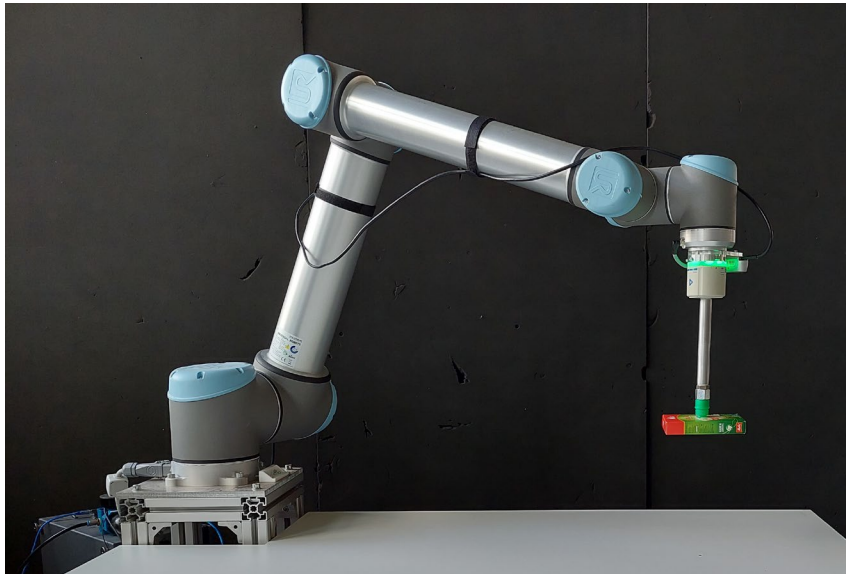


- Definition des zu betrachtenden Systems / Festlegung der Modellgrenzen
- Für die Simulation nicht benötigte Komponenten und Aspekte werden vernachlässigt oder substituiert.
- Definition der erforderlichen Eingangs- und Ausgangsgrößen
- Unterteilung des Modells in einzelne Teilmodelle, die zueinander über bekannte Schnittstellen in Beziehung stehen
- Auswahl von geeigneten Simulationsverfahren

Für das betrachtete Beispiels eines Handhabungsroboters kann das Simulationsmodell durch vier miteinander in Beziehung stehende Teilmodelle dargestellt werden.

Steuerungssoftware

- Sensordatenverarbeitung, Objekterkennung
- Greifposebestimmung
- Trajektorienplanung
- Ablaufsteuerung



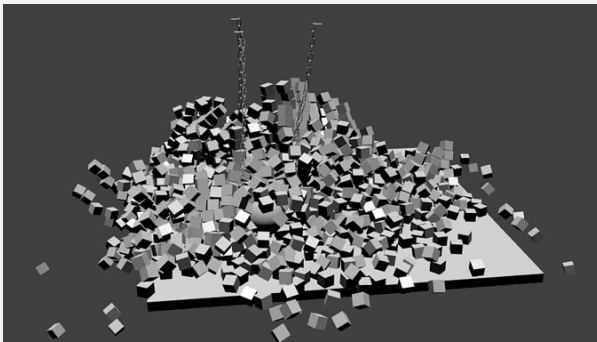
Mehrkörpersimulation

Sensorsimulation

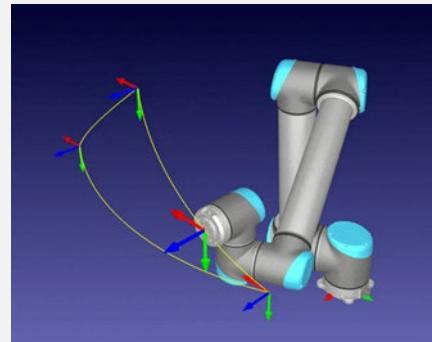
Steuerungssimulation

Zur Simulation der Bewegung eines starren Körpers wird dessen Bewegung durch Differentialgleichungen beschrieben, die mittels numerischer Verfahren gelöst werden.

- Reale physikalische Phänomene wie Bewegungen können zur mathematischen Darstellung in Form von Differentialgleichungen beschrieben werden.
- Mittels numerischer Verfahren können diese Differentialgleichungen gelöst werden, um eine zeitdiskrete Beschreibung des zugrundeliegenden Phänomens zu erhalten.



Bildquelle: cgtrader.com



Bildquelle: nicholasnadeau.me

Benötigt werden somit die allgemeinen Zustandsdifferentialgleichungen, die die Bewegung eines starren Körpers beschreiben, in der Form:

$$\dot{x} = f(t, x) \quad \text{wobei } t = \text{Zeitverlauf der Simulation}$$

- Ohne vorhandene äußere Einflüsse in Form von Kräften oder Drehmomenten befindet sich ein starrer Körper in seiner Ruhelage, seine Zustände ändern sich in diesem Fall nicht.
- Durch einwirkende Kräfte und Momente wird die aktuelle Pose, Geschwindigkeit und Beschleunigung geändert.
- Die am Schwerpunkt eines Körpers angreifenden Kräfte und Momente können durch die beiden folgenden Größen beschrieben werden.

Kräfte und Momente am Massenschwerpunkt

Kräfte $f_{S,0} = \begin{pmatrix} F_{S,x} \\ F_{S,y} \\ F_{S,z} \end{pmatrix}$

Drehmomente $n_{S,K} = \begin{pmatrix} \tau_{S,x} \\ \tau_{S,y} \\ \tau_{S,z} \end{pmatrix}$

Bezogen auf den
Schwerpunkt

Im Körperkoordinatensystem K
(bzw. Weltkoordinatensystem 0)

Der starre Körper wird über seine Pose, seine Masse und Trägheitsmomente, die auf ihn wirkenden Kräfte- und Momente sowie die resultierenden Geschwindigkeiten beschrieben.

Basierend auf den Kenntnissen aus VE 05 bzw. den bekannten Prinzipien aus der Vorlesungen „Dynamik starrer Körper“ und „Mathematik für Ingenieure“ lässt sich ein starrer Körper durch folgende Eigenschaften grundlegend beschreiben:

Pose des starren Körpers

Translation $c_{S,0} = \begin{pmatrix} x_s \\ y_s \\ z_s \end{pmatrix}$ Rotation $r = \begin{pmatrix} \varphi \\ \psi \\ \theta \end{pmatrix}$
(as XYZ-Euler-Winkel)

Linear- und Drehgeschwindigkeit

Lineargeschwindigkeit

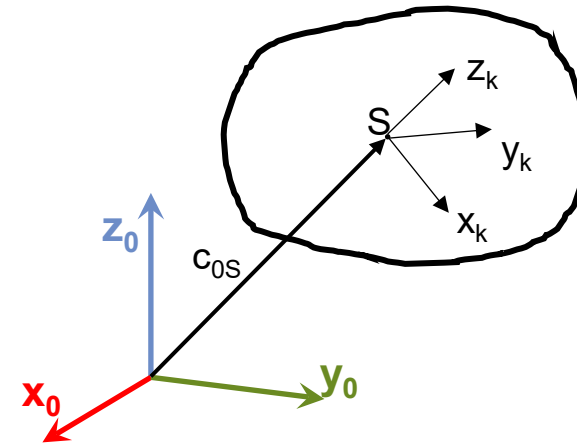
$$v_{S,0} = \begin{pmatrix} v_{S,x} \\ v_{S,y} \\ v_{S,z} \end{pmatrix}$$

Drehgeschwindigkeit

$$\omega_{S,K} = \begin{pmatrix} \omega_{S,x} \\ \omega_{S,y} \\ \omega_{S,z} \end{pmatrix}$$

Masse und Trägheitsmoment

Masse m Hauptträgheitsmomente $I = \begin{bmatrix} I_x & 0 & 0 \\ 0 & I_y & 0 \\ 0 & 0 & I_z \end{bmatrix}$



Somit ergibt sich der Zustandsvektor x und die daraus folgende Zustandsdifferentialgleichung zu

$$x = \begin{pmatrix} x_s \\ y_s \\ z_s \\ \varphi \\ \psi \\ \theta \\ v_{S,x} \\ v_{S,y} \\ v_{S,z} \\ \omega_{S,x} \\ \omega_{S,y} \\ \omega_{S,z} \end{pmatrix} \quad \dot{x} = f(t, x) \rightarrow \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\varphi} \\ \dot{\psi} \\ \dot{\theta} \\ \dot{v}_{S,x} \\ \dot{v}_{S,y} \\ \dot{v}_{S,z} \\ \dot{\omega}_{S,x} \\ \dot{\omega}_{S,y} \\ \dot{\omega}_{S,z} \end{pmatrix} = f \left(t, \begin{pmatrix} x_s \\ y_s \\ z_s \\ \varphi \\ \psi \\ \theta \\ v_{S,x} \\ v_{S,y} \\ v_{S,z} \\ \omega_{S,x} \\ \omega_{S,y} \\ \omega_{S,z} \end{pmatrix} \right)$$

Durch Auflösen der Newton-Euler-Gleichungen unter Berücksichtigung der zugrundeliegenden KOSYs ergeben sich die zugehörigen Zustandsgleichungen.

Basierend auf den Newton-Euler-Gleichung können die Differentialgleichungen für die einzelnen Zustandsgrößen bestimmt werden

Newton-Euler Gleichung
(vgl. VE 05)

$$\begin{pmatrix} n_c \\ f \end{pmatrix} = \begin{pmatrix} I\dot{\omega} + \omega \times I\omega \\ m\ddot{c} \end{pmatrix}$$

$$\dot{x} = f(t, x) \rightarrow \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\phi} \\ \dot{\psi} \\ \dot{\theta} \\ \dot{v}_{S,x} \\ \dot{v}_{S,y} \\ \dot{v}_{S,z} \\ \dot{\omega}_{S,x} \\ \dot{\omega}_{S,y} \\ \dot{\omega}_{S,z} \end{pmatrix} = f \left(t, \begin{pmatrix} x_S \\ y_S \\ z_S \\ \phi \\ \psi \\ \theta \\ v_{S,x} \\ v_{S,y} \\ v_{S,z} \\ \omega_{S,x} \\ \omega_{S,y} \\ \omega_{S,z} \end{pmatrix} \right)$$

Lösen der Differentialgleichung mittels numerischer Verfahren, um die Bewegung des starren Körpers unter Einfluss von Kräften und Moment zu bestimmen

$$\begin{aligned} \dot{x} &= v_{S,x} \\ \dot{y} &= v_{S,y} \\ \dot{z} &= v_{S,z} \end{aligned}$$

$$\begin{pmatrix} \dot{\phi} \\ \dot{\psi} \\ \dot{\theta} \end{pmatrix} = {}^k_0R \begin{bmatrix} \omega_{S,x} \\ \omega_{S,y} \\ \omega_{S,z} \end{bmatrix}$$

(Wobei k_0R die Rotation des starren Körpers
Im Weltkoordinatensystem beschreibt,)

$$\dot{v}_x = \frac{F_{S,x}}{m}$$

$$\dot{v}_y = \frac{F_{S,y}}{m}$$

$$\dot{v}_z = \frac{F_{S,z}}{m}$$

$$\dot{\omega}_{S,x} = \frac{I_y - I_z}{I_x} \omega_{S,y} \omega_{S,z} + \frac{\tau_{S,x}}{I_x}$$

$$\dot{\omega}_{S,y} = \frac{I_z - I_x}{I_y} \omega_{S,x} \omega_{S,z} + \frac{\tau_{S,y}}{I_y}$$

$$\dot{\omega}_{S,z} = \frac{I_x - I_y}{I_z} \omega_{S,x} \omega_{S,y} + \frac{\tau_{S,z}}{I_z}$$

Numerische Verfahren erlauben die approximative Lösung von Differentialgleichungen, die analytisch nicht oder nur sehr aufwändig gelöst werden können.

Grundlegendes zur Lösung von gewöhnlichen Differentialgleichungen

- Gewöhnliche Differentialgleichungen hängen nur von einer Veränderlichen ab, haben somit die Form

$$\dot{x}(t) = f(t, x)$$

- Viele Differentialgleichungen lassen sich nicht analytisch lösen, sodass ihre Lösung alternativ durch numerische Lösungsverfahren approximiert werden muss (wird auch als numerische Integration bezeichnet).
- Die Approximation kann dabei beliebig genau werden, und ist von der gewählten Schrittweite des Lösungsverfahrens abhängig.
- Numerische Integrationsverfahren können auch höchstgenaue Lösungen berechnen, erfordern in diesem Fall jedoch viel Rechenleistung.
- In der Praxis muss daher ein Kompromiss zwischen Genauigkeit der Lösung und Echtzeitverhalten des Lösungsverfahrens getroffen werden.
- Zur numerischen Lösung einer Differentialgleichung muss

Numerische Integration nach dem Euler-Verfahren

Das einfachste Verfahren zur numerischen Lösung gewöhnlicher Differentialgleichungen ist das Eulerverfahren, das sich aus der Taylorreihenentwicklung ableitet.

Taylorreihenentwicklung einer Funktion $x(t)$ um den Punkt $x_k = x(t_k)$:

$$x(t) = x(t_k) + \dot{x}(t_k)(t - t_k) + \frac{1}{2}\ddot{x}(t_k)(t - t_k)^2 + \dots + R_n(t)$$

Euler-Verfahren mit Integrationsschrittweite $t - t_k = \Delta t$:

$$x(t_k + \Delta t) = x(t_k) + \underbrace{\dot{x}(t_k)\Delta t}_{\text{Term des Euler-Verfahrens}} + \underbrace{\frac{1}{2}\ddot{x}(t_k)\Delta t^2 + \dots + R_n(t)}_{\text{Fehlerterm}}$$

Unter Vernachlässigung des Fehlerterms



$$x(t_k + \Delta t) = x(t_k) + \dot{x}(t_k)\Delta t$$

Das klassische Runge-Kutta-Verfahren liefert im Vergleich zum Euler-Verfahren eine deutlich höhere Genauigkeit und kommt daher auch in der Praxis zum Einsatz.

Numerische Integration nach dem Runge-Kutta Verfahren

- Da der betragsmäßig hohe Fehlerterm des Euler-Verfahrens zu signifikanten Abweichungen führen kann, werden in der Praxis alternative Verfahren eingesetzt.
- Beim Runge-Kutta-Verfahren wird die Lösung für den aktuellen Zeitschritt auf Basis mehrerer Stützstellen bestimmt, um den verbleibenden Fehler zu minimieren.

Beispiel:

Gleichung des 4-stufigen Runge-Kutta-Verfahrens

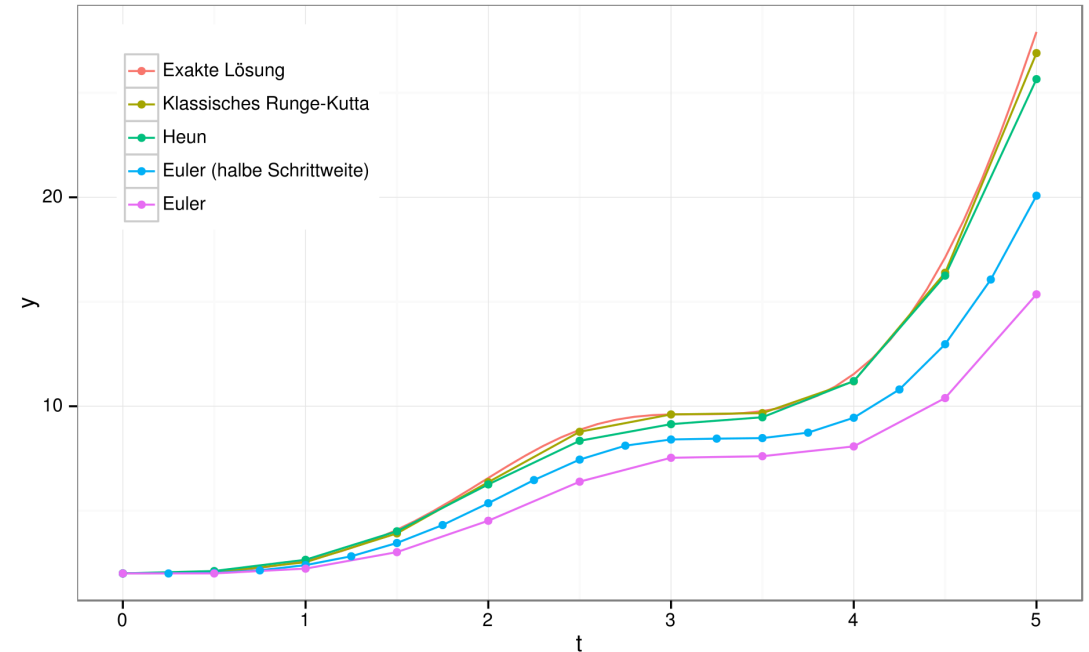
$$K_1 = f[t_k, x(t_k)]$$

$$K_2 = f\left[t_k + \frac{\Delta t}{2}, x(t_k) + \frac{\Delta t}{2} K_1\right]$$

$$K_3 = f\left[t_k + \frac{\Delta t}{2}, x(t_k) + \frac{\Delta t}{2} K_2\right]$$

$$K_4 = f[t_k + \Delta t, x(t_k) + \Delta t * K_1]$$

➔
$$x(t_k + \Delta t) = x(t_k) + \frac{\Delta t}{6} (K_1 + 2K_2 + 2K_3 + K_4)$$



Result of numerical integration methods for the function $y' = \sin(t)^2 y$, image source: Wikipedia

- Das klassische Euler-Verfahren wird aufgrund der hohen Abweichungen in der Praxis nur selten eingesetzt.
- Das Runge-Kutta-Verfahren liefert deutlich bessere Ergebnisse und kommt in verschiedenen Ausführungsformen auch in bestehenden Softwaretools zur Simulation zum Einsatz.

Für einfache Gleichungssysteme kann die numerische Lösung noch ohne den Einsatz von Simulationsprogrammen bestimmt werden.

Das grundlegende Vorgehen zur Lösung einer gewöhnlichen Differentialgleichung ist somit wie folgt:

- 1 Festlegen der Anfangswerte

$$x(t_k = 0) = x_0$$

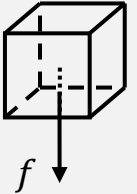
sowie der Schrittweite Δt .

- 2 Berechnung der approximierten Lösung für den nächsten Zeitpunkt $t_k + \Delta t$ mittels eines numerischen Verfahrens

- 3 Erhöhung von t_k um einen Zeitschritt Δt und Wiederholung von Schritt 2

Beispiel:
Starrer Körper im freien Fall

$$\dot{x}_z = v_z \quad \dot{v}_z = \frac{F_z}{m} \quad m = 1 \text{ kg} \quad f = \begin{pmatrix} 0 \\ 0 \\ -9.81 \end{pmatrix}$$



Anfangswerte

$$x(t_k = 0) = 0$$

$$\dot{v}_z(t_k = 0) = -9.81 \frac{\text{m}}{\text{s}^2}$$

$$v_z(t_k = 0) = 0$$

$$\Delta t = 0.1 \text{ s}$$

Geschwindigkeit

$$v_z(\Delta t) = v_z(0) + \dot{v}_z(0)\Delta t = 0 - 9.81 \frac{\text{m}}{\text{s}^2} * 0.1 \text{ s} = -0.98 \frac{\text{m}}{\text{s}}$$

$$v_z(2\Delta t) = v_z(\Delta t) + \dot{v}_z(\Delta t)\Delta t = 0.98 \frac{\text{m}}{\text{s}} - 9.81 \frac{\text{m}}{\text{s}^2} * 0.1 \text{ s} = -1.96 \frac{\text{m}}{\text{s}}$$

$$v_z(3\Delta t) = v_z(2\Delta t) + \dot{v}_z(2\Delta t)\Delta t = 1.96 \frac{\text{m}}{\text{s}} - 9.81 \frac{\text{m}}{\text{s}^2} * 0.1 \text{ s} = -2.94 \frac{\text{m}}{\text{s}}$$

Position

$$x_z(\Delta t) = x_z(0) + \dot{x}_z(0)\Delta t = 0 - 0 = 0 \text{ m}$$

$$x_z(2\Delta t) = x_z(\Delta t) + \dot{x}_z(\Delta t)\Delta t = 0 - 0.98 \frac{\text{m}}{\text{s}} * 0.1 \text{ s} = -0.09 \text{ m}$$

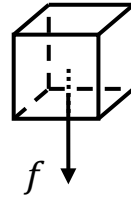
$$x_z(3\Delta t) = x_z(2\Delta t) + \dot{x}_z(2\Delta t)\Delta t = -0.09 \text{ m} - 1.96 \frac{\text{m}}{\text{s}} * 0.1 \text{ s} = -0.29 \text{ m}$$

Basierend auf den Zustandsgleichungen lässt sich das Verhalten eines starren Körpers unter Einwirkung externer Kräfte und Momente simulieren, jedoch ist noch keine Interaktion möglich.

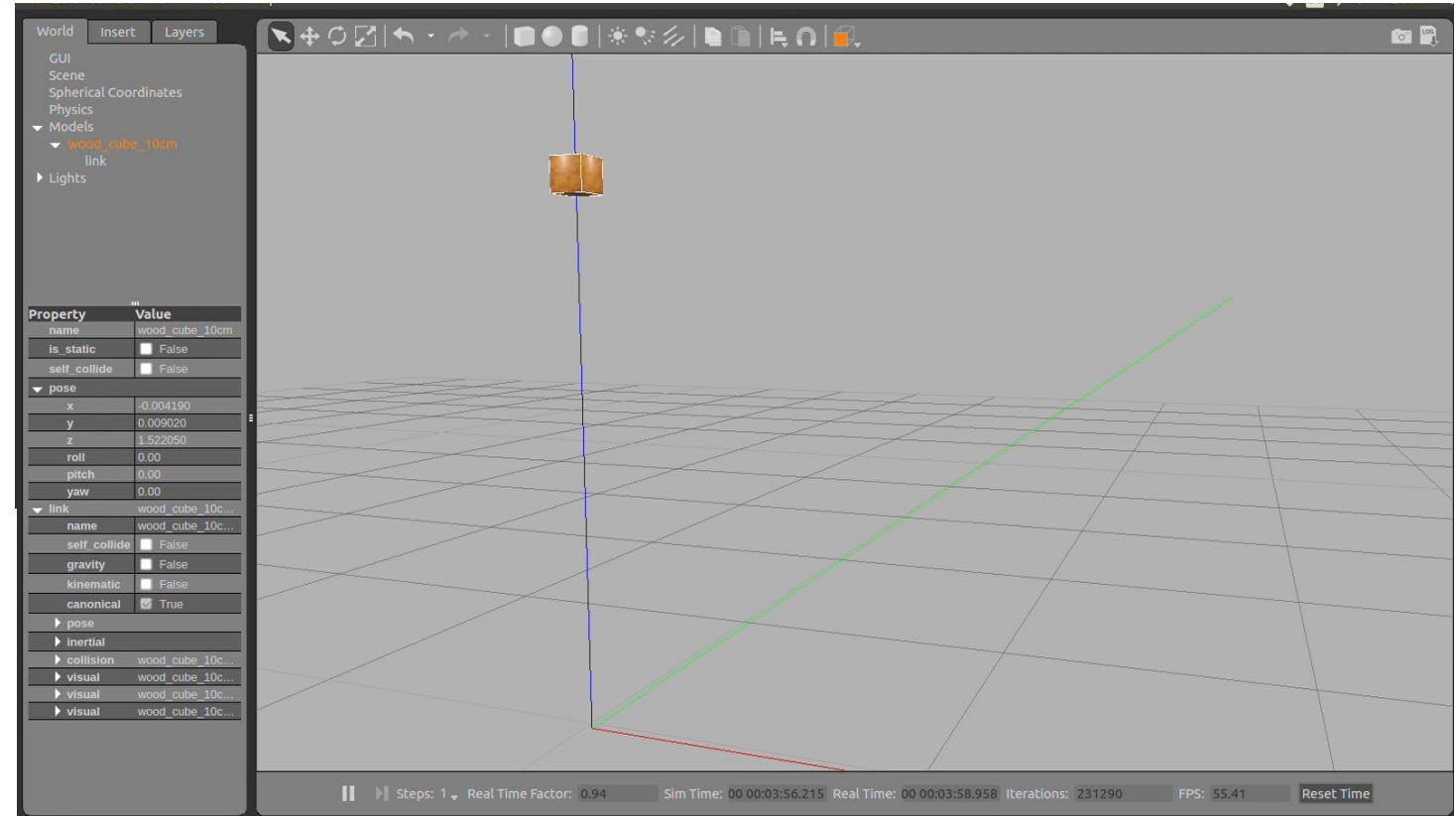
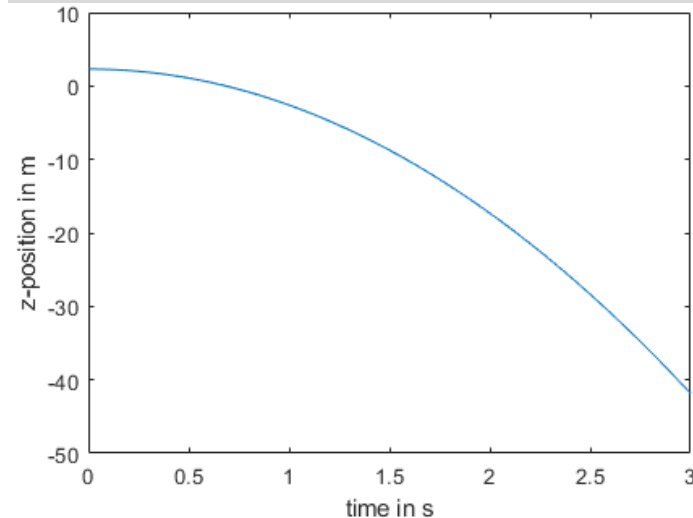
Freier Fall eines Quaders

Anfangsbedingungen

$$\begin{aligned} R &= 0 & m &= 1 \text{ kg} \\ v &= 0 & c &= \begin{pmatrix} 0 \\ 0 \\ 2.3 \end{pmatrix} \\ \omega &= 0 \\ n_c &= 0 & f &= \begin{pmatrix} 0 \\ 0 \\ -9.81 \end{pmatrix} \end{aligned}$$



Verlauf der globalen z-Position
des Würfels im freien Fall



- Ein Roboter setzt sich in der Realität aus mehreren Körpern zusammen, die über definierte Gelenke und Lager miteinander in Verbindung stehen.
- Erweiterung der Simulation zur Mehrkörpersimulation erforderlich**

Zur Simulation komplexer Robotersysteme muss die Starrkörpersimulation noch um den Bereich der Mehrkörpersimulation (MKS) ergänzt werden.

Merkmale und Eigenschaften der Mehrkörpersimulation

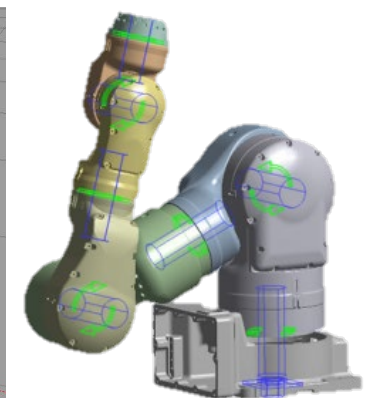
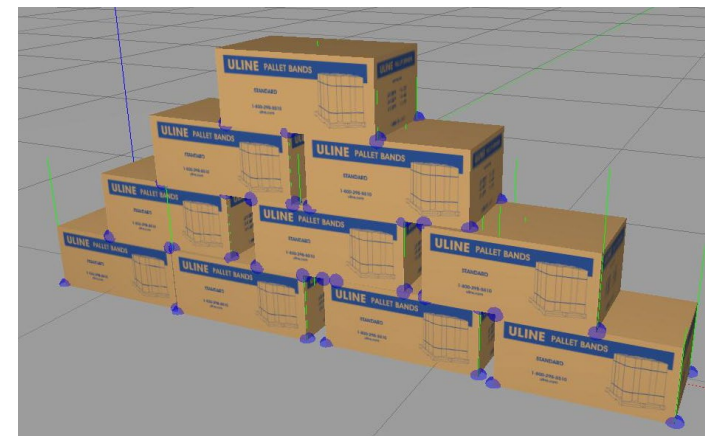
- Verknüpfung von starren, massenbehafteten Körpern durch masselose Verbindungselemente
- Die Verbindungselemente können dabei die Bewegung der Körper limitieren sowie die Kraft- und Momentübertragung zwischen den einzelnen Körpern beeinflussen.
- Die physikalischen Eigenschaften der Verbindungselemente werden in den zugrundeliegenden Systemdifferentialgleichungen berücksichtigt.

Der Einsatz der Mehrkörpersimulation (MKS) ermöglicht die Simulation komplexer kinematischer Ketten wie beispielsweise Knickarmroboter oder Antriebsstränge, aber:

- Bei falscher Wahl der Berechnungsparameter, können die Ergebnisse stark von der Realität abweichen.
- Hoher Rechenleistungsbedarf, insbesondere bei komplexen Modellen und vielen Körpern

Die Verbindungselemente der MKS können beispielsweise folgenden Einflüsse darstellen:

- Limitierung von Drehwinkeln
- Einbringung von Lagerkräften / Reduktion der Freiheitsgrade der Bewegungen / Darstellung der Nachgiebigkeit von Lagern
- Berücksichtigung von Haft-, Gleit- oder Rollreibungen
- Berücksichtigung von Dämpfungen
- Darstellung von Kontaktelementen zur Kraftübertragung



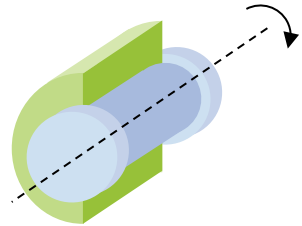
Bildquelle: Ansys

Die bei der Mehrkörpersimulation eingesetzten Verbindungselemente lassen sich grundlegend in kinematische und elastische Bindungen unterteilen.

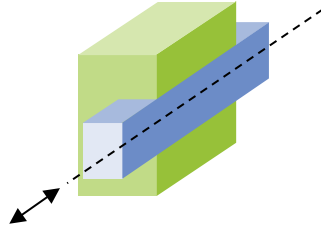
Kinematische Bindungen

Limitieren die Bewegungsfreiheitsgrade starrer Körper

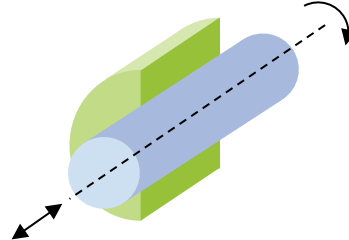
Examples



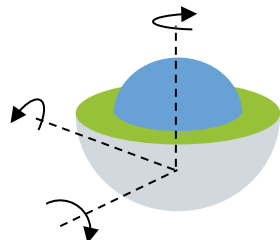
Drehgelenk
(revolute joint)



Schubgelenk
(prismatic joint)



Zylindergelenk
(cylindrical joint)



Kugelgelenk
(ball joint)



Festlager
(fixed bearing)

Elastische Bindungen

Limitieren die Bewegungsfreiheitsgrade eines Körpers nicht

Beispiele

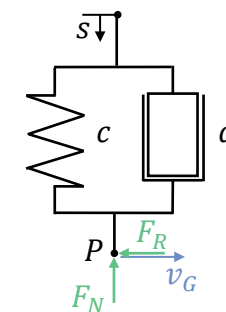
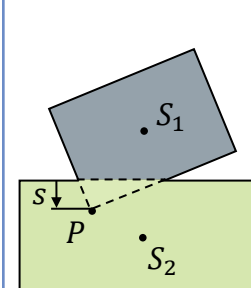
■ Punkt-zu-Punkt Kraftelemente

- z.B. Seile, Federn, pneumatische und elektrische Stellglieder
- Modellierung als Feder-Dämpfer-System

■ Kontaktelemente

- Ermöglichen die Übertragung von Kontaktkräften bei Kollisionen
- Erlaubt die Berücksichtigung von Oberflächeneigenschaften

Vereinfachtes Modell des Kontaktelementes



$$s = \text{Eindringtiefe}$$

$$v_G = \text{Gleitgeschwindigkeit}$$

$$F_N = cs + d\dot{s} \quad \text{für } s \geq 0 \quad \text{mit } |d\dot{s}| < cs$$

$$F_R = -\mu F_N \frac{v_G}{|v_G|}$$

Model based on: Rill, G., Schaeffer, T.: „Grundlagen und Methodik der Mehrkörpersimulation“, Springer, 2017; ISBN 978-3-658-16008-1

Zur Berechnung von Berührungspunkten und den resultierenden Kontaktkräften muss die Simulation um geeignete Algorithmen zur Kollisionserkennung erweitert werden.

Bedeutung der Kollisionserkennung und –behandlung für Robotersimulationen

- Mit Hilfe der Kollisionserkennung werden Überschneidungen zwischen einzelnen geometrischen Objekten ermittelt.
- Zur Kollisionserkennung können verschiedene geometrische Hüllkörper zum Einsatz kommen, die sich hinsichtlich Genauigkeit und Rechenzeit unterscheiden.
- Sobald eine Kollision und die daraus resultierende Überlappung zweier Körper berechnet wurde, können die hieraus resultierenden Kontaktkräfte ermittelt werden.
- Insbesondere bei komplexen Geometrien und Umgebungen mit vielen Objekten kann die Kollisionsberechnung einen signifikanten Anteil der Berechnungszeit einnehmen.

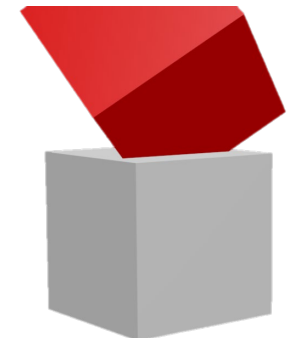
Bei der Kollisionsbehandlung können zwei grundlegende Ansätze unterschieden werden:

Kontinuierliche Kollisionserkennung

In jedem Zeitschritt der Simulation wird basierend auf den zugrundeliegenden physikalischen Regeln berechnet, welche Objekte sich im nachfolgenden Zeitschritt überlappen bzw. berühren werden. Die Kollisionsbehandlung kann somit im Voraus angewendet werden.

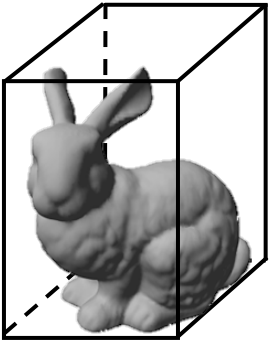
Diskrete Kollisionserkennung

In jedem Zeitschritt der Simulation wird eine Liste der sich berührenden bzw. überlappenden Objekte berechnet. Auf diese Objekte wird anschließend (nachträglich) die Kollisionsbehandlung (inkl. einer Korrektur der Überlappung) angewendet.

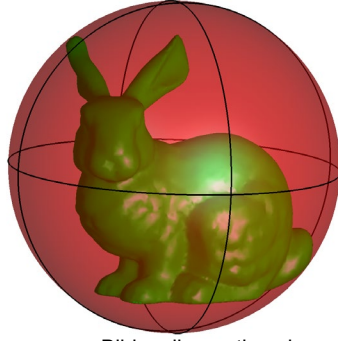


Zur Kollisionsprüfung können verschiedene Hüllkörper zum Einsatz kommen, die sich hinsichtlich Genauigkeit und Berechnungsaufwand deutlich unterscheiden.

Primitiver Hüllkörper



bounding box



Bildquelle: mathworks.com

bounding sphere

- Einfache geometrische Hüllkörper, die eine rudimentäre Kollisionsprüfung ermöglichen
- Präzise Ergebnisse können nur für einfache Geometrien erreicht werden
- Geringste Komplexität der Kollisionsberechnung

Konvexe Hüllkörper

(engl. discrete oriented polytope - *DOP*)

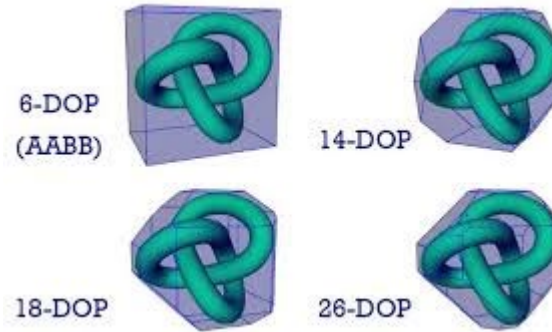


Image: inf.elte.hu

- k-DOP's erlauben mehrere, paarweise parallel zueinander liegende Beschränkungsflächen, wodurch sie Objekte enger einschließen können.
- Höhere Genauigkeit als primitive Hüllkörper, dennoch oftmals eine zu starke Vereinfachung der Realität
- Mittlere Komplexität

Polygonnetze

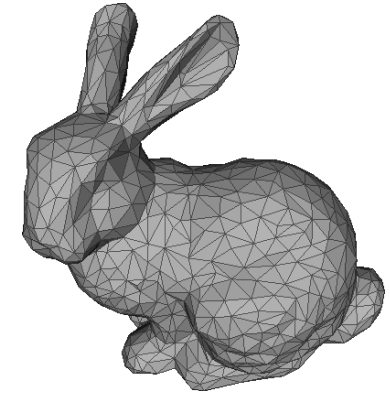


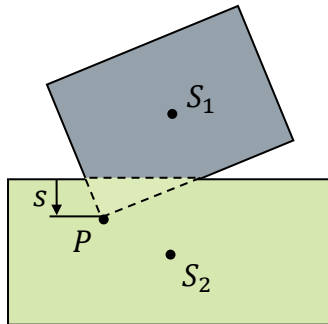
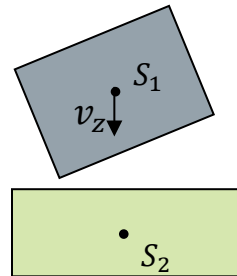
Image: polytechnique.fr

- Polygonnetze ermöglichen eine präzise Kollisions- und Kontaktberechnung basierend auf der realen Geometrie des Körpers.
- Hohe Komplexität und im Vergleich deutlich längere Rechenzeiten
- Berechnungsaufwand stark von der Netzauflösung abhängig

Basierend auf den berechneten Kontaktpunkten zwischen den gewählten Hüllkörpern kann der bei einer Kollision übertragene Impuls bestimmt werden.

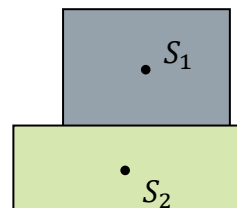
Basierend auf der berechneten Durchdringung der Körper bzw. deren Relativgeschwindigkeiten und ihren Masseeigenschaften können die resultierenden Kontaktkräfte bestimmt werden.

Körper 1 fällt unter Einfluss der Schwerkraft vertikal nach unten



Kollision der Hüllkörper wird erfasst, die bei der Kollision übertragenen Kräfte werden berechnet und die nicht zulässige Überschneidung der Körper wird korrigiert

Sobald ein Gleichgewichtszustand erreicht ist, kommt Körper 1 auf Körper 2 zum Liegen.



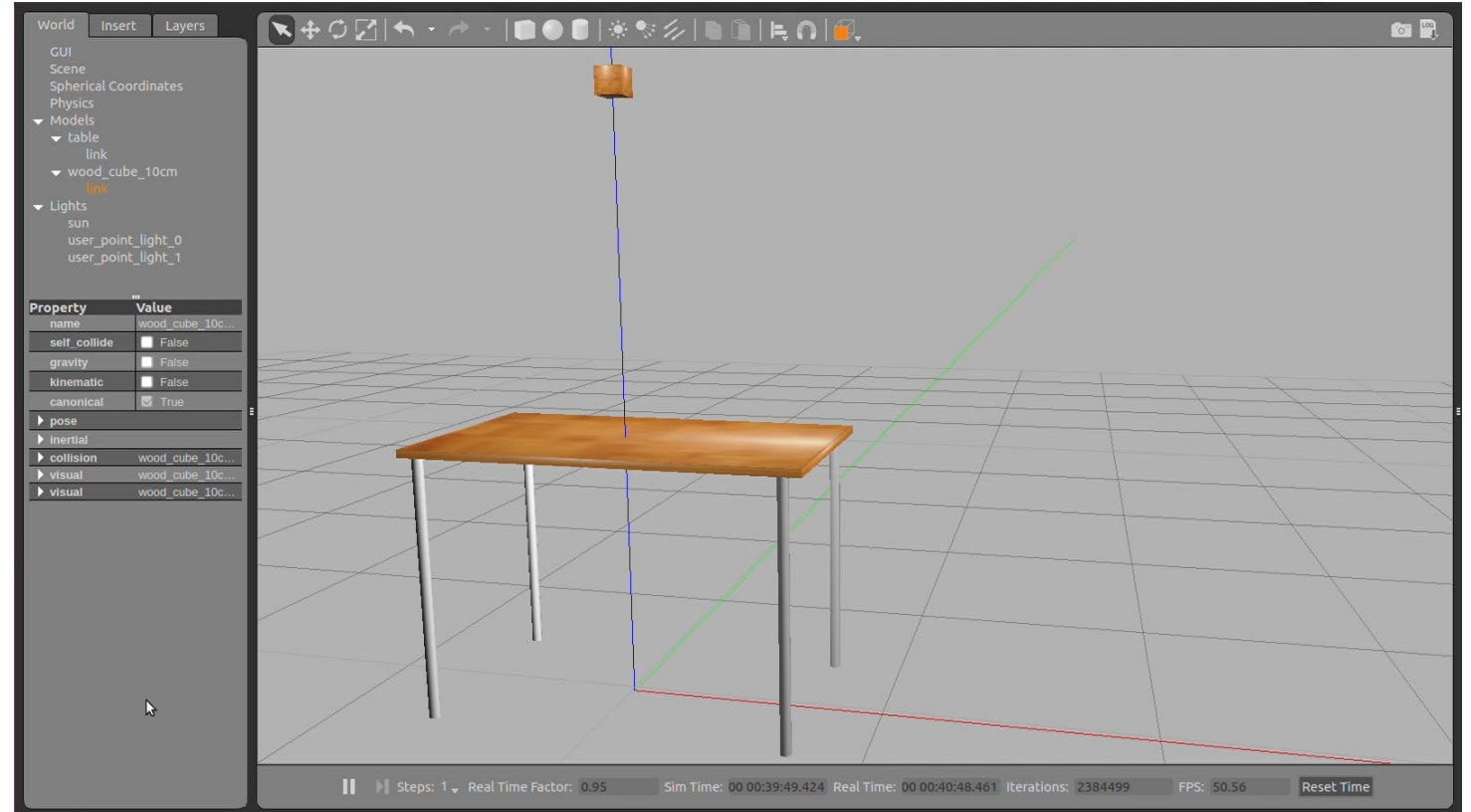
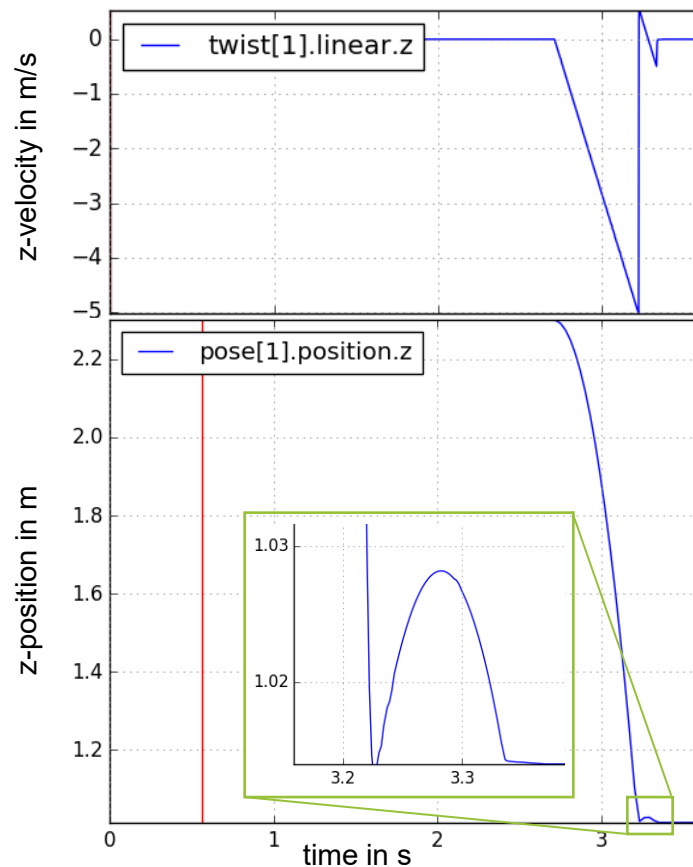
Mögliches Vorgehen bei der Berechnung von Kollisionen und Kontaktkräften in Simulationen

- 1 Zur Bestimmung der Kontaktkräfte werden vor jedem Zeitschritt der Simulation die Berührungspunkte (oder Berührungsflächen) aller kollidierenden Körper bestimmt.
- 2 An allen berechneten Kollisionspunkten wird ein Kontaktobjekt angelegt, welches das Übertragen eines Geschwindigkeitsvektors / Impulses von einem Objekt auf ein anderes Objekt ermöglicht.
- 3 Diese Kontaktobjekte enthalten neben den aktuellen Zustandsvektoren und Masseninformationen der kollidierenden Objekte auch Informationen bzgl. Reibungskoeffizienten und Oberflächenelastizität.
- 4 Der Simulationsschritt wird durchgeführt und die zugrundeliegenden Differentialgleichungen zur Bestimmung der Zustandsvektoren werden berechnet.
- 5 Die angelegten Kontaktobjekte werden wieder entfernt.

Durch die Integration einer geeigneten Kollisionsprüfung wird eine physikalisch korrekte Simulation der Umgebungsinteraktion ermöglicht.

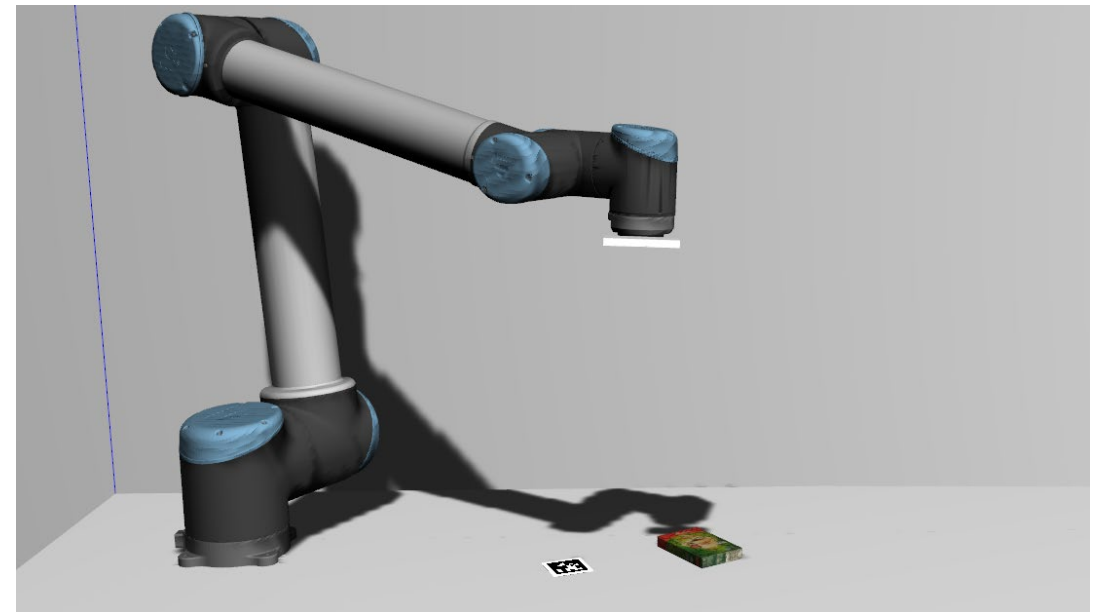
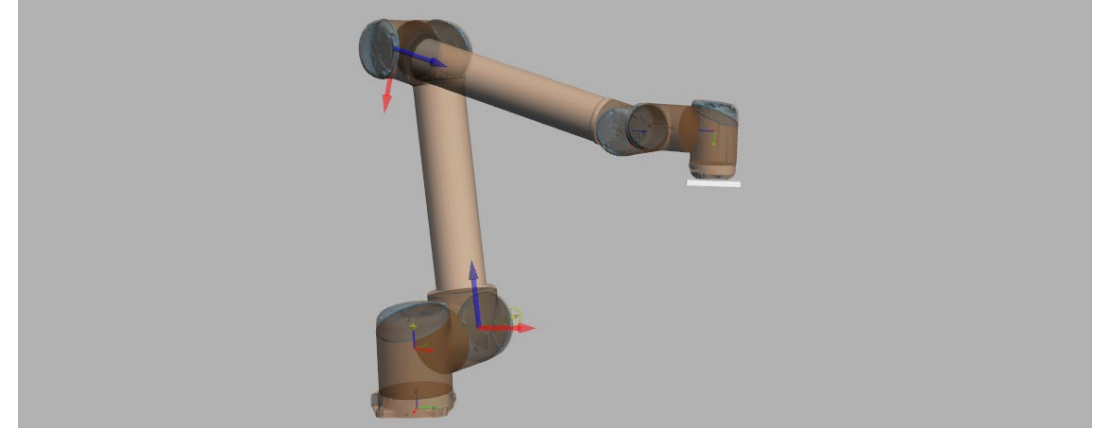
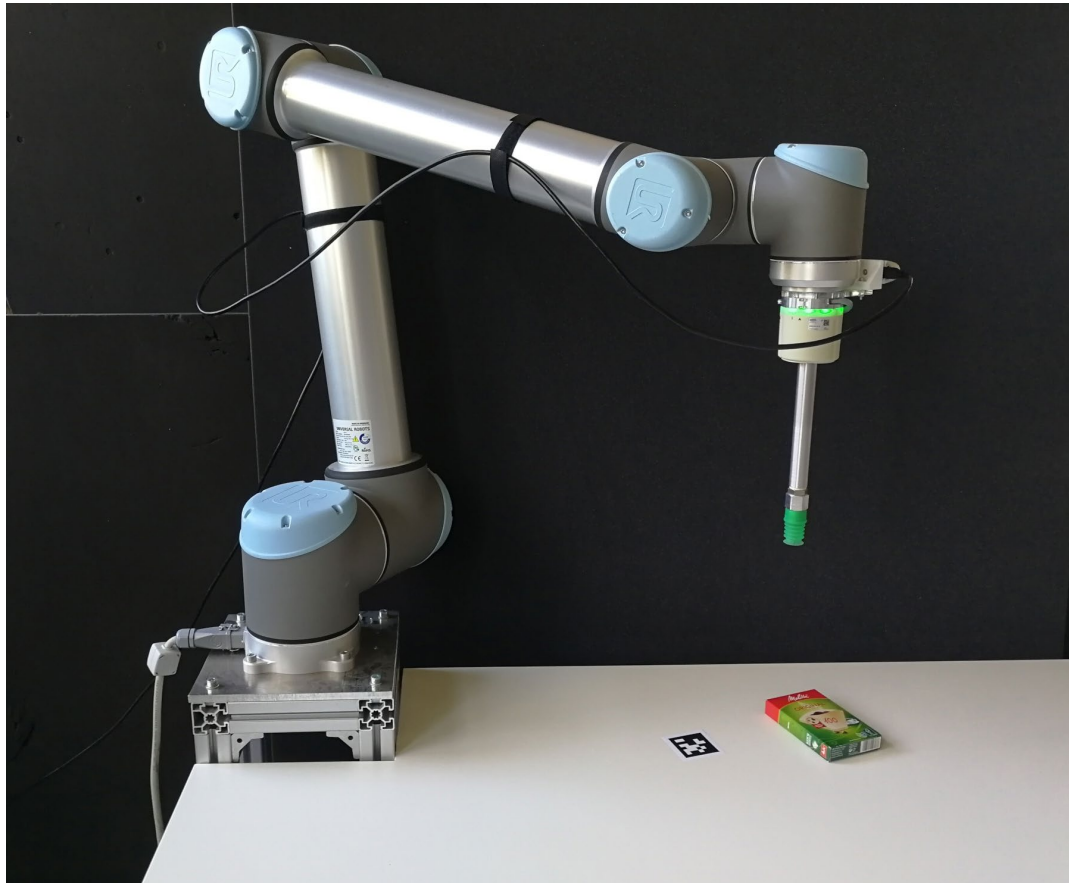
Freier Fall eines Quaders mit Kollisionsbehandlung

(Anfangsbedingungen wie zuvor)



- In der Praxis ist es nicht effizient, die erforderlichen Gleichungssysteme für jede Simulation neu herzuleiten und zu implementieren.
- Durch den Einsatz bestehender Simulationsframeworks lässt sich die Entwicklungszeit deutlich reduzieren.

Ausgehend vom realen Modell kann ein kinematisches Modell des Roboters aufgebaut und dessen Verhalten mittels Mehrkörpersimulation simuliert werden.



Um Simulationsmodelle analog zur Realität bewegen zu können, werden Motoren simuliert, die in der Lage sind, externe Kräfte und Momente in die Simulation einzubringen.

- Auf die simulierten Gelenke können analog zur Realität Kräfte und Drehmomente einwirken.
- Um einen Motor abzubilden, muss die aus der überlagerten Regelung vorgegebenen Soll-Drehgeschwindigkeit in eine Ist-Drehzahl sowie Drehmoment umgewandelt werden.
- Je nach erforderlicher Simulationstiefe und verfügbaren Eingangsdaten und Modellinformationen können verschiedene Ansätze zur Simulation von Motoren zum Einsatz kommen.

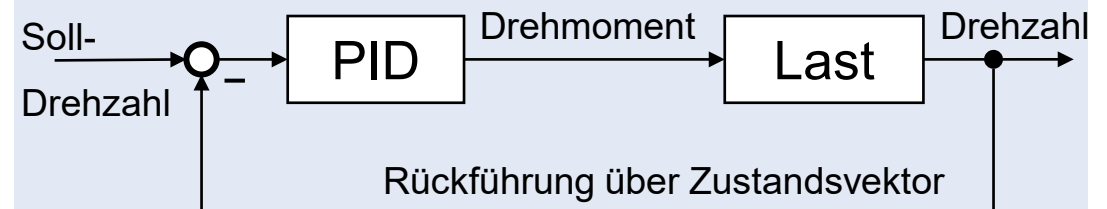
Direktes Setzen der berechneten Soll-Geschwindigkeit über Schnittstelle der Physiksimulation

```
model->GetJoint("joint_wrist_1")->SetVelocity(0, 1.0);
```

Ebenfalls direkt über ROS-Topics möglich

- Erreichen der Soll-Geschwindigkeit in nur einem Simulationsschritt
- Kein Überschwingen / keine Anlaufphase
- Benötigt keine zusätzliche Rechenleistung
- Bildet das reale physikalische System nicht ab

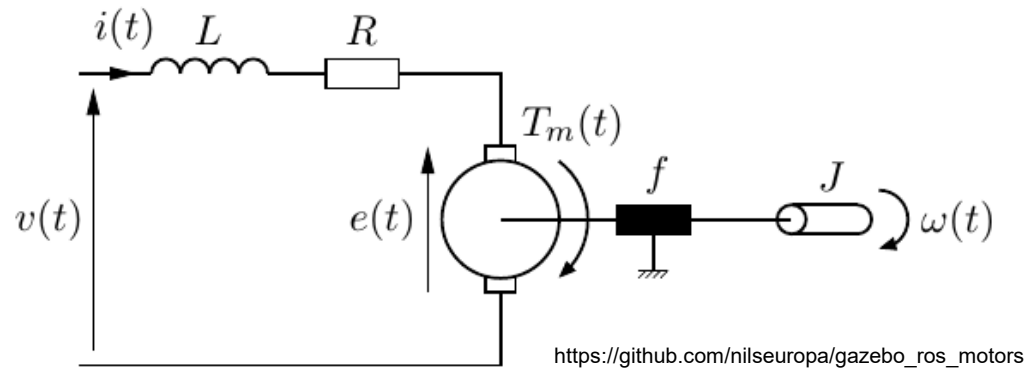
Simulierter Gelenkmotor als PID-Regler



http://gazebosim.org/tutorials?tut=set_velocity&cat=#SetJointVelocityUsingPIDcontrollers

- Ermöglicht die realitätsnähere Simulation eines Motors inklusive Überschwingen und Anlaufverhalten
- Erfordert das individuelle Einstellen der Koeffizienten
- Benötigt zusätzliche Rechenleistung bei der Simulation
- Bildet das reale physikalische System nur bedingt ab

Sind zusätzliche Informationen über den Motortyp sowie dessen Kenndaten verfügbar, kann die Simulation auch auf Basis eines elektro-mechanischen Modells erfolgen.



Mathematischer Zusammenhang

Kirchoff's Gesetz

$$L \frac{di(t)}{dt} = v(t) - Ri(t) - e(t)$$

Newton's Gesetz

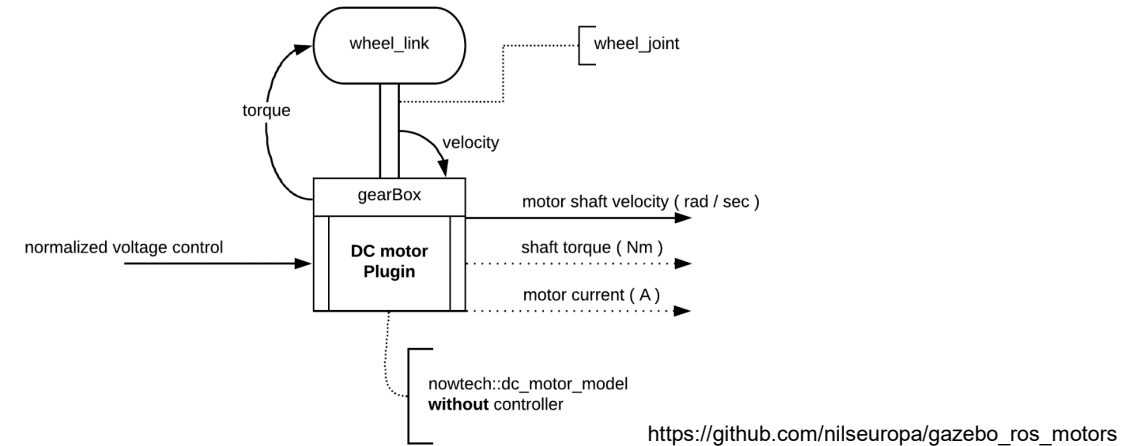
$$J \frac{d\omega(t)}{dt} = \sum T(t) = T_m(t) - f\omega(t)$$

Elektromechanische Kopplung

$$e(t) = K_e \omega(t)$$

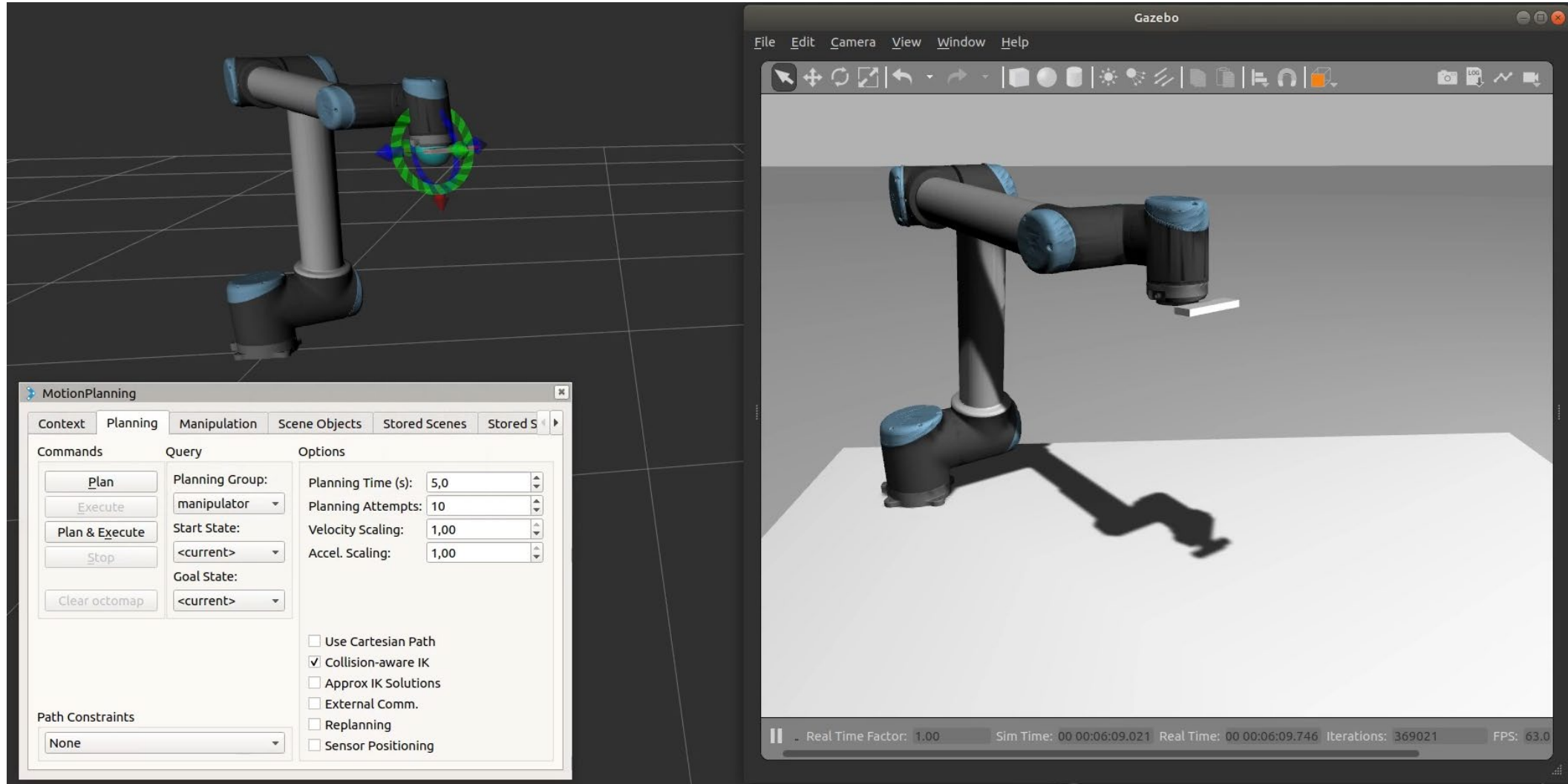
$$T_m(t) = K_T i(t)$$

	Variables	Parameters
Electrical dynamics	$i(t)$: current (A) $v(t)$: voltage (V) $e(t)$: back EMF (V)	R : Resistance (Ω) L : Inductance (H)
Mechanical dynamics	$T_m(t)$: Motor Torque (N.m) $\omega(t)$: Angular velocity (rad.s^{-1})	f : friction torque (N.m.s.rad^{-1}) J : Moment of inertia (kg.m^2).

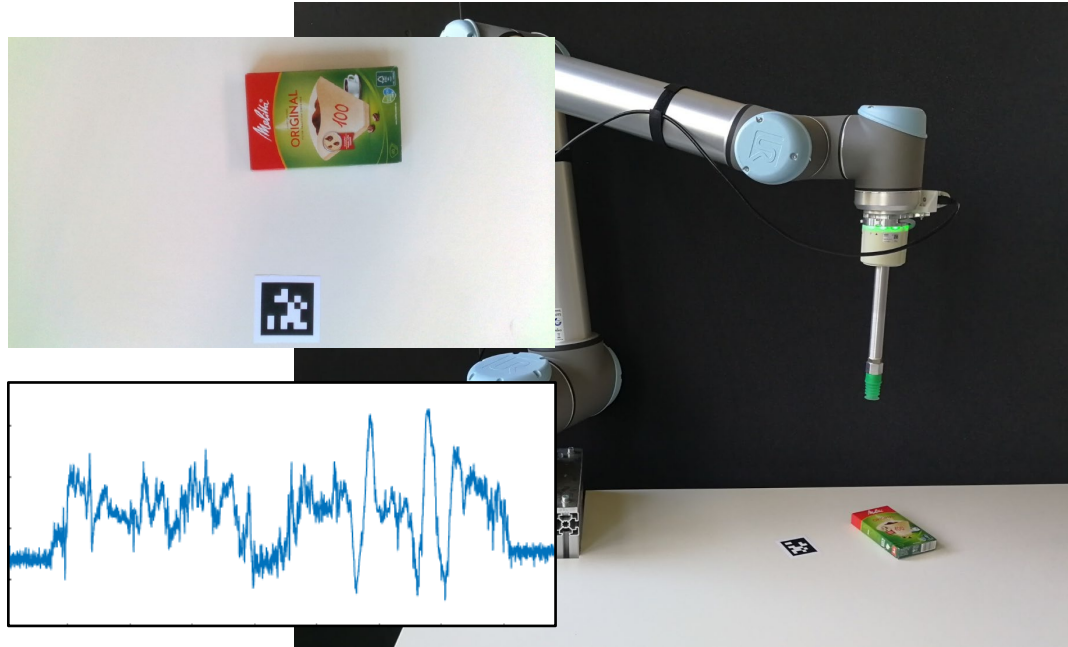


- Ermöglicht die realitätsnahe Simulation eines Motors inklusive Überspringen und Anlaufverhalten
- Ermöglicht eine Simulation der Motors auf Basis der realen Eingangsgröße (el. Spannung im Fall eines DC-Motors)
- Erfordert detaillierte Informationen über den Motor
- Benötigt zusätzliche Rechenleistung bei der Simulation

Sowohl für den realen als auch den simulierte Roboter können über das ROS2-Package MoveItTrajektorien geplant und über die identische Schnittstelle ausgeführt werden.



Leistungsfähige Simulationsumgebungen ermöglichen neben der Mehrkörpersimulation auch eine realitätsnahe Umgebungsdarstellung und die Simulation von Sensoren.



- Wie im Realsystem sind auch in der Simulation Sensordaten notwendig, um ein Robotersystem zu steuern und zu regeln.
- Zu Simulation von Sensoren stehen verschiedene Vorgehensweisen und Programme zur Verfügung.
- Je genauer ein Sensor simuliert werden kann, desto besser kann die Simulation das Realsystem wiedergeben.

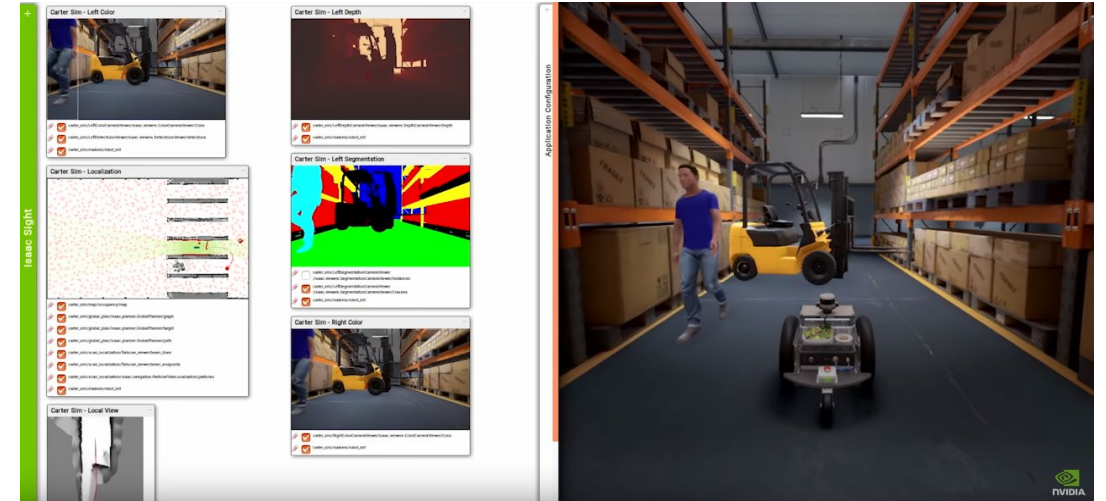
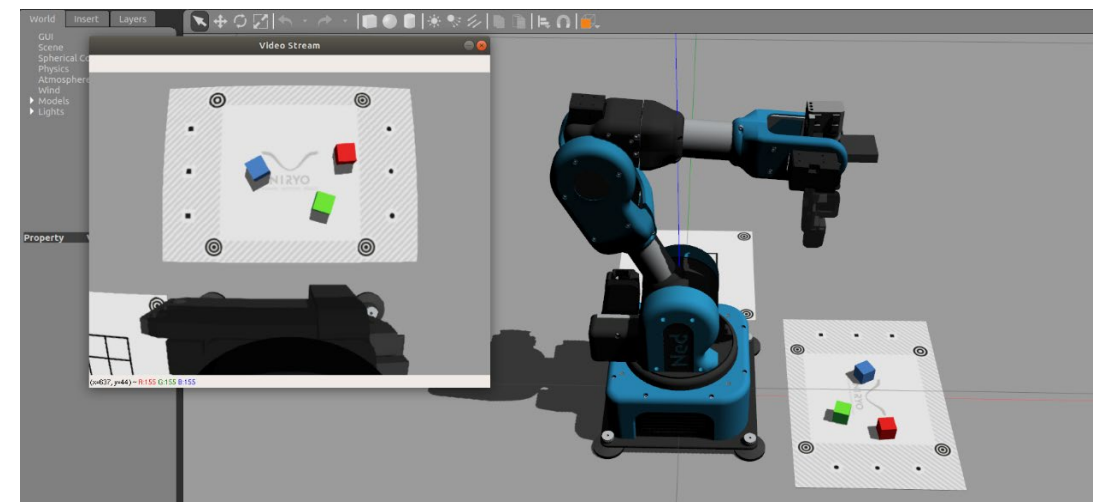


Image: Niryo

Image: NVIDIA

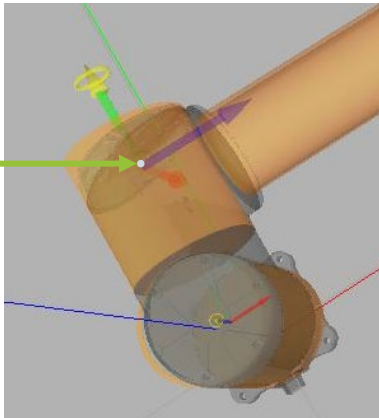


Zur Simulation von Sensoren, die Zustandsgrößen messen, können die benötigten Werte direkt aus der Physiksimulation bezogen und nachträglich mit Rauschen ergänzt werden.

Simulation von Drehzahl- und Drehwinkelsensoren

Zur Simulation dieser Sensoren kann direkt auf das Zustandsmodell des zugehörigen Verbindungselements zugegriffen werden. Gilt analog auch für lineare Positions- und Geschwindigkeitssensoren.

Simulation



Direkter Zugriff auf die Werte des simulierten Zustandsvektor x .

Realität



Bildquelle: Universal Robots

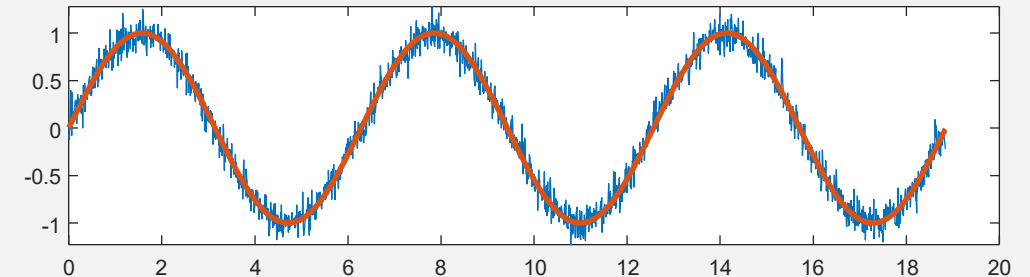
Erfassung des Gelenkwinkels über Magnetencoder

Da reale Sensoren Messabweichungen aufweisen und Größen teilweise nur indirekt messen können, müssen diese Störungen auch in der Simulation berücksichtigt werden.

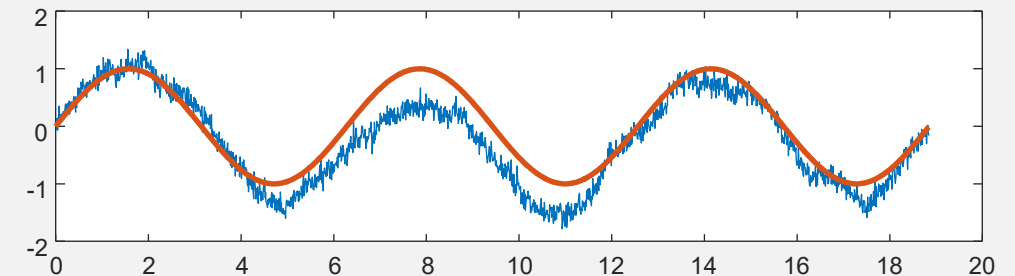
Addition von Rauschen

Da die Simulation auf idealisierten Modellen beruht, können zusätzliche Störeinflüsse hinzugefügt werden, um reale, physikalische Vorgänge besser abbilden zu können. Z.b:

- Weißes Rauschen zur Abbildung von stochastischen Messabweichungen



- Kontinuierliche Abweichungen, z.B. zur Abbildung von Temperatureinflüssen



Zur Simulation von Kameras wird eine virtuelle Kamera angelegt, für die analog zur Bildschirmausgabe die Grafikpipeline durchlaufen wird.

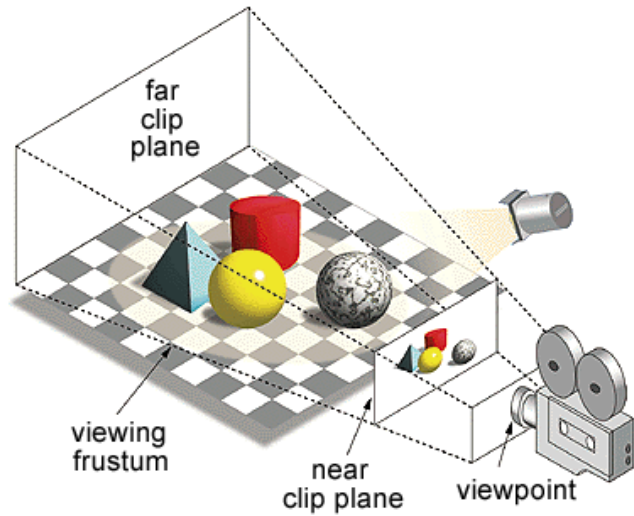


Image: Unity.com

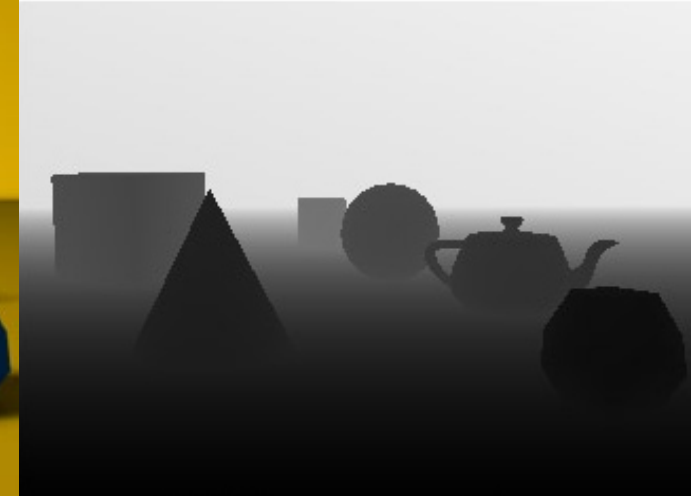
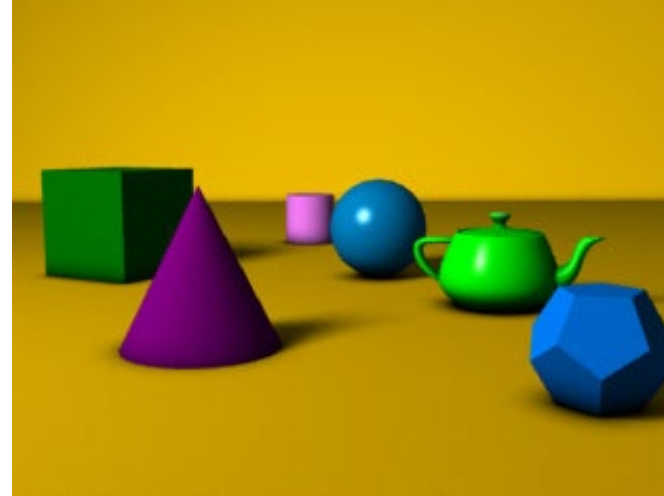


Image: wikipedia.org

2D Kamera

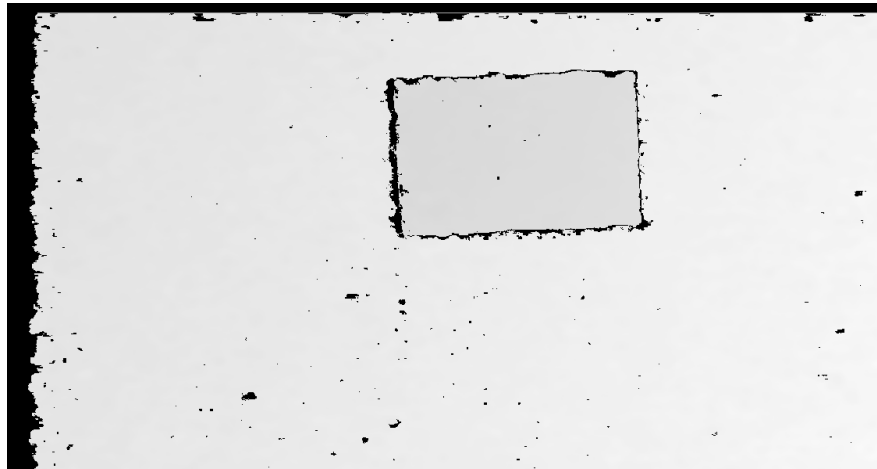
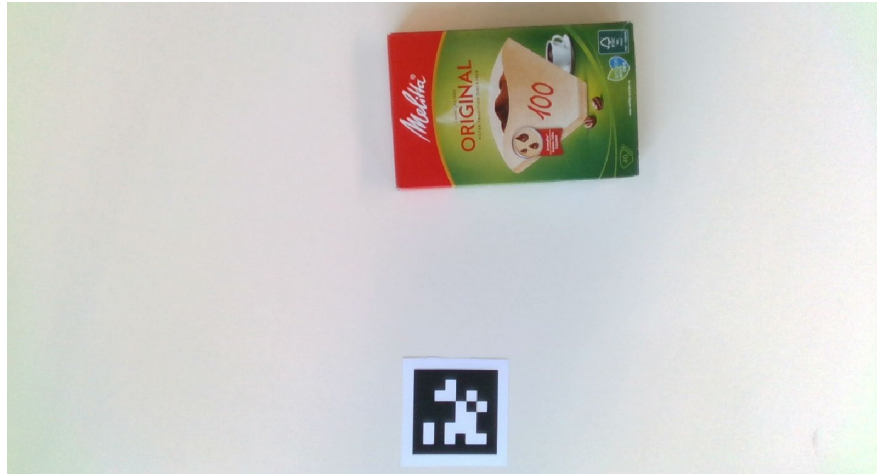
- Zur Simulation einer (Farb-)Kamera wird eine virtuelle Kamera angelegt, deren Ausgabebild über die Grafikpipeline berechnet wird.
- Die Berechnung erfolgt dabei analog zur Berechnung des auf dem Bildschirm ausgegebenen Bildes.
- Zur Simulation der Kamera müssen deren optische Eigenschaften wie Auflösung, Öffnungswinkel bzw. Brennweite oder Verzerrungskoeffizienten vorab festgelegt werden.

3D Kamera

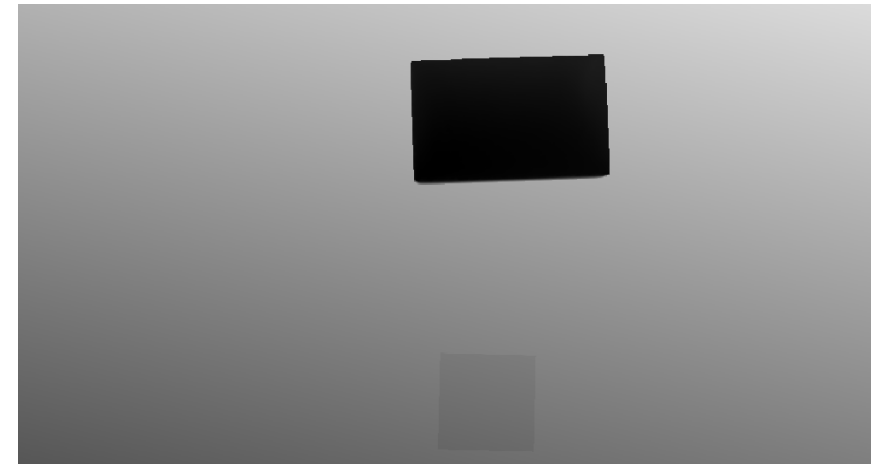
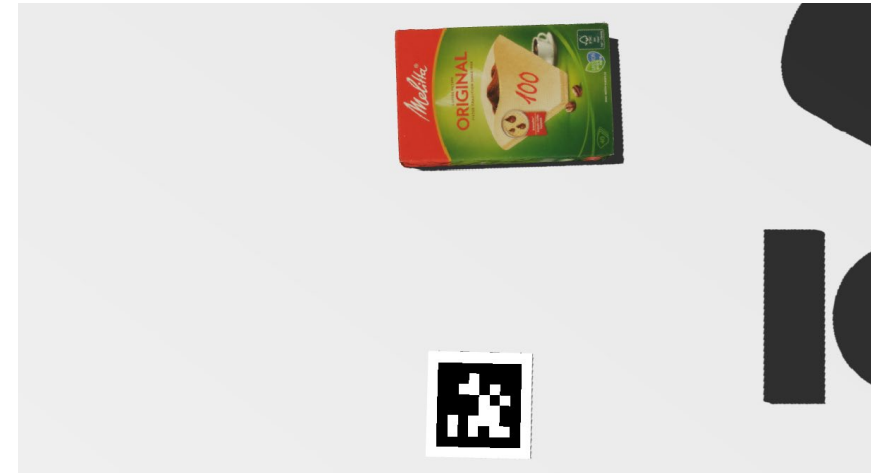
- Zur Simulation einer Tiefenkamera wird zusätzlich der Z-Buffer der Grafikpipeline herangezogen.
- Der Z-Buffer enthält den pixelweisen Abstand zwischen der Kamera und allen von ihr sichtbaren Objekte (vgl. Bild oben rechts).
- Das Ausgabebild einer simulierten Tiefenkamera entspricht somit dem Z-Buffer der Grafikkarte, der mit Rauschen überlagert und an die Kameraparameter der simulierten Tiefenkamera angepasst wurde.

Der Vergleich von simulierten und realen Kamerabildern zeigt insbesondere bei den Tiefeninformationen, dass das Rauschen des echten Sensors falsch modelliert wurde.

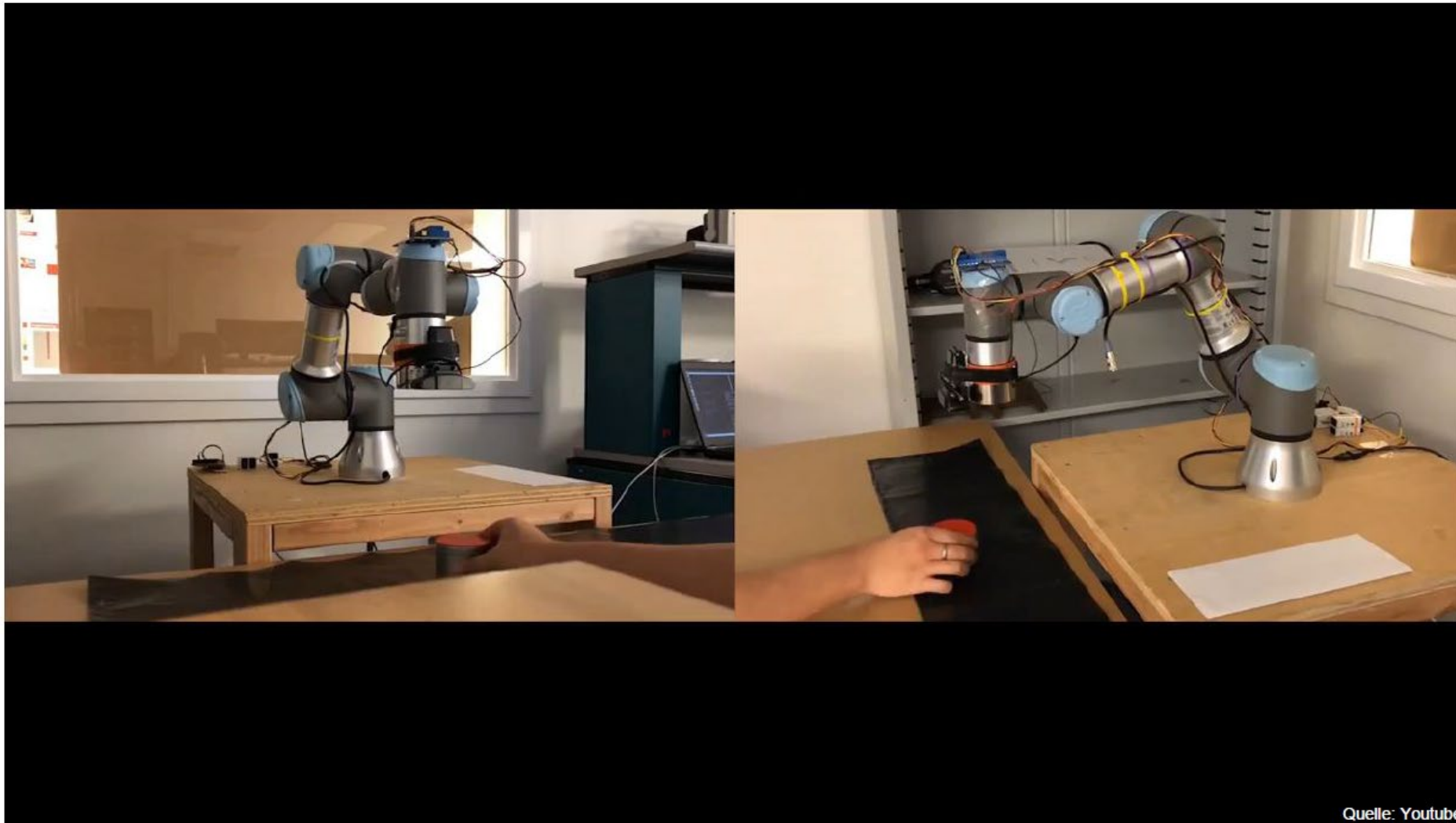
Reale Sensordaten (Farb- und Tiefenbild)



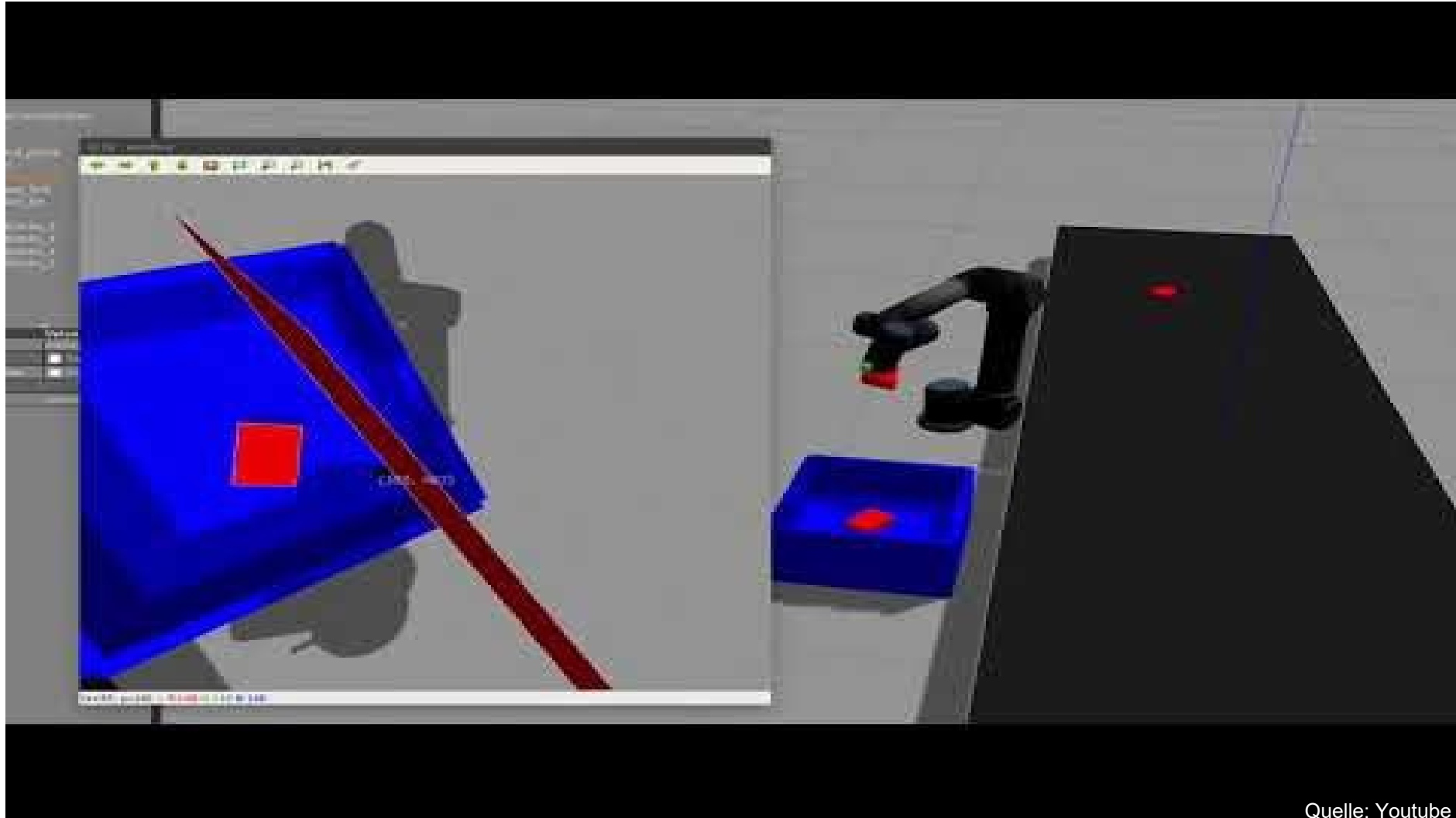
Simulierte Sensordaten (Farb- und Tiefenbild)



Durch die Simulation von Handhabungsaufgaben können potentielle Kollisionen erkannt werden und die reale Roboterhardware wird geschont.

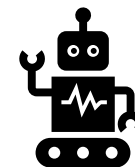
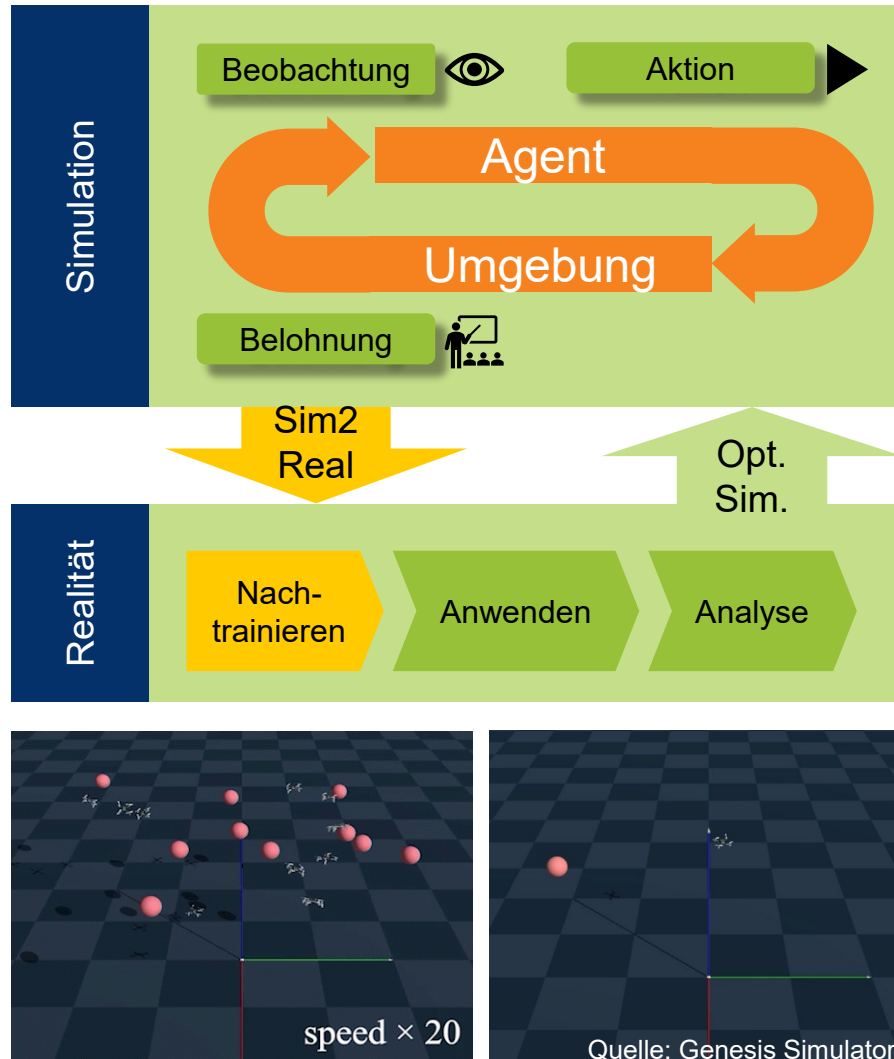


Durch die Simulation von Handhabungsaufgaben können potentielle Kollisionen erkannt werden und die reale Roboterhardware wird geschont.



Quelle: Youtube

Simulatoren können auch dazu dienen, um auf Reinforcement Learning basierende Agenten zu trainieren und nach Überwindung der Sim2Real Gap in der Realität anzuwenden.



Die Verifikation und die Validierung von Simulationen stellen sicher, dass die gewonnen Erkenntnisse auf die Realität übertragbar sind.

Simulationen basieren stets auf einem abstrahierten (vereinfachten) Modell der Wirklichkeit. Die Ergebnisse einer Simulation sollen Rückschlüsse auf das wahre Verhalten des Originals ermöglichen. Daher ist sicherzustellen, dass das Simulationsmodell korrekt und valide ist.

Verifikation

Überprüfung, ob das Simulationsmodell das konzeptionelle Modell korrekt wiedergibt, also richtig umgesetzt wurde

- Zentrale Frage: **Ist das Modell richtig?**
- Bei komplexen Simulationsmodellen nachträglich oft nicht vollständig überprüfbar
- Muss bereits während der Modellentwicklung berücksichtigt, und kontinuierlich geprüft werden

Validierung

- Überprüfung, ob das Simulationsmodell ausreichend mit dem Originalsystem übereinstimmt, also für dessen Simulation geeignet ist
- Zentrale Frage: **Ist es das richtige Modell?**
- Durch direkten Vergleich erzielter Ergebnisse in Realität und Simulation überprüfbar
- Eingangsdaten müssen bekannt und selbst valide sein
- Problematisch, da Vergleich oft nicht möglich
 - Originalsystem existiert (noch) nicht
 - Versuchslauf am Originalsystem würde Simulation ersetzen

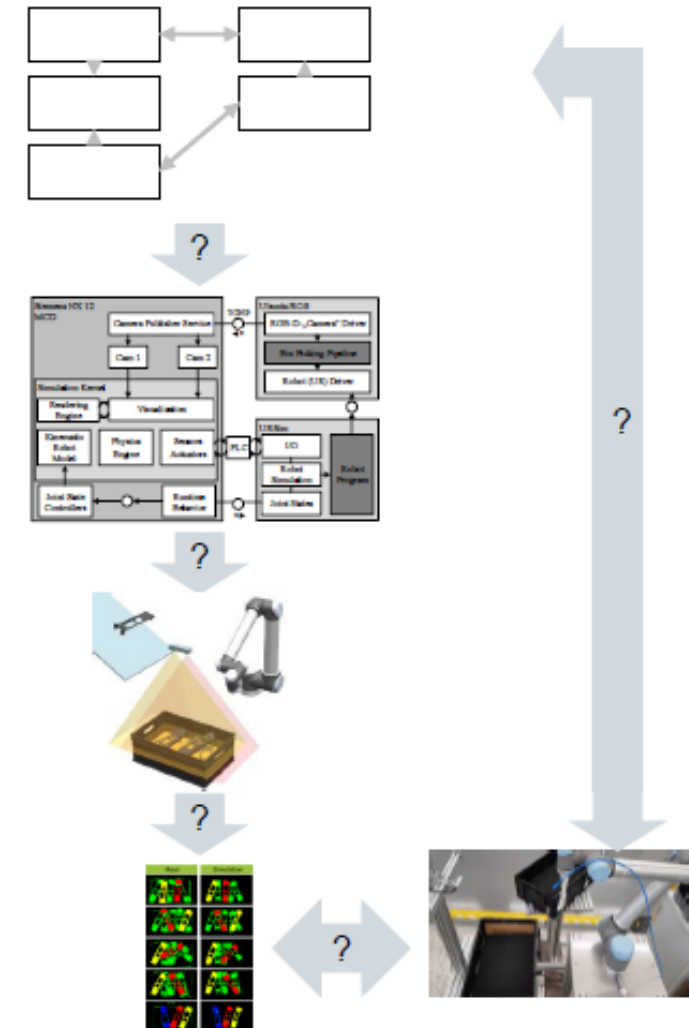
Es existierend zahlreiche Techniken zur Verifizierung und Validierung, die sich hinsichtlich ihrer Aussagekraft und dem benötigtem Aufwand unterscheiden.

- Animation
- Begutachtung (Review)
- Dimensionstest
- Ereignis Validitätstest
- Festwerttest
- Grenzwerttest
- Monitoring
- Schreibtischtest
- Sensitivitätsanalyse
- Statistische Techniken
- Strukturiertes Durchgehen
- Trace-Analyse
- Turing-Test
- Vergleich mit anderen Modellen
- Vergleich mit aufgezeichneten Daten

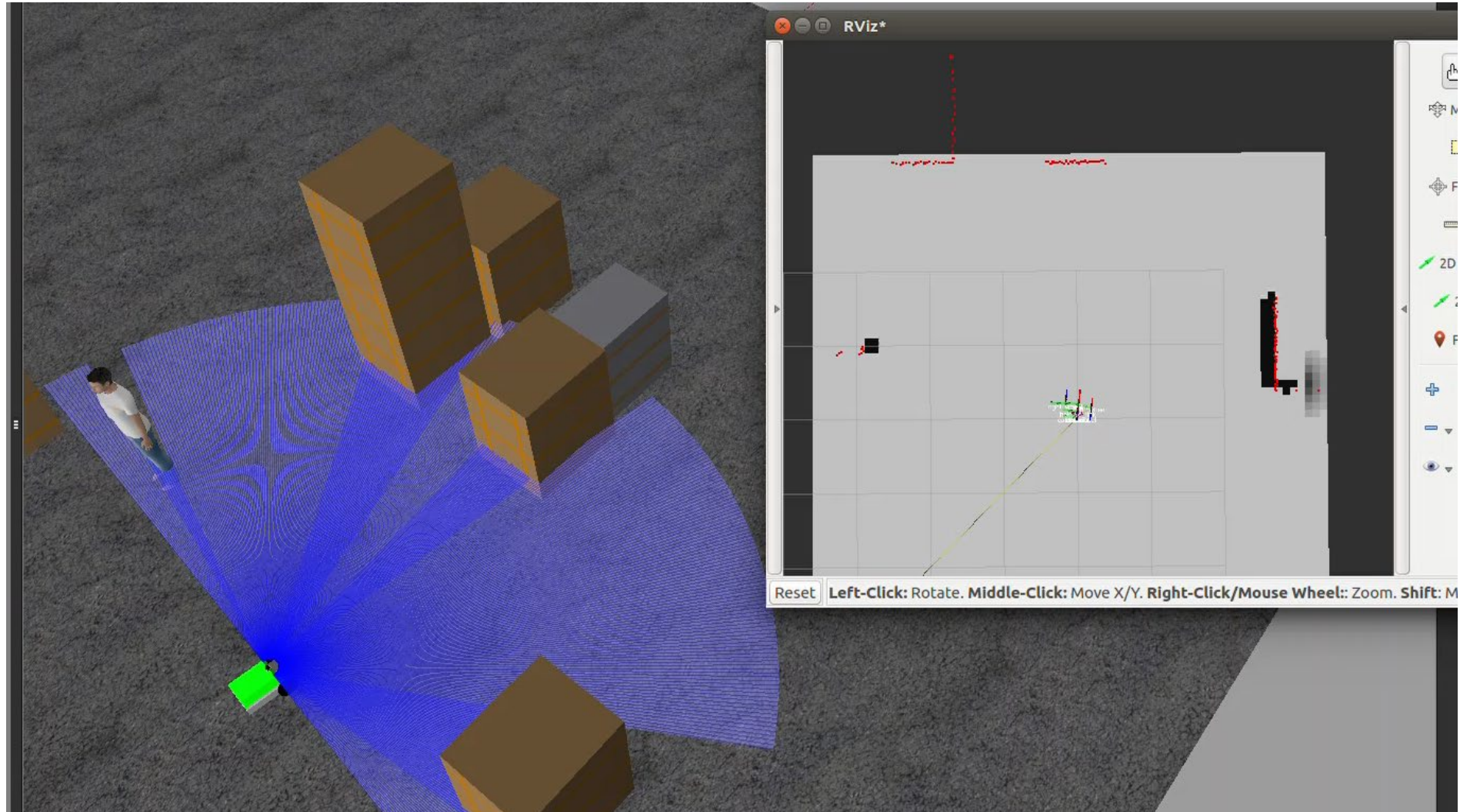
...

Quelle: Rabe et al. (2008)

- Verschiedenste Techniken können zur Verifizierung und Validierung eingesetzt werden.
- Die Testverfahren sind oftmals simulations- und einsatzspezifisch.
- Meist erfolgt die Verifizierung durch ein systematisches Prüfen der Implementation
- Die Validierung erfolgt
 - durch Expertenbeurteilung
 - durch den Vergleich mit analytisch berechenbaren Daten (an möglichen Punkten)
 - durch den Vergleich mit bekannten (historischen) oder ähnlichen (vergleichend) Daten



Im Rahmen der Übung werden wir ein Simulationsmodell eines mobilen Roboters entwickeln, im Simulationsframework Gazebo aufbauen und mit ROS ansteuern.



The following literature is recommended to deepen the contents of this lecture unit:

Bossel, H: „Systeme, Dynamik, Simulation: Modellbildung, Analyse und Simulation komplexer Systeme“, 2004; ISBN 978-3-833-40984-4

Rill, G., Scheaffer, T.: „Grundlagen und Methodik der Mehrkörpersimulation“, Springer, 2017; ISBN 978-3-658-16008-1

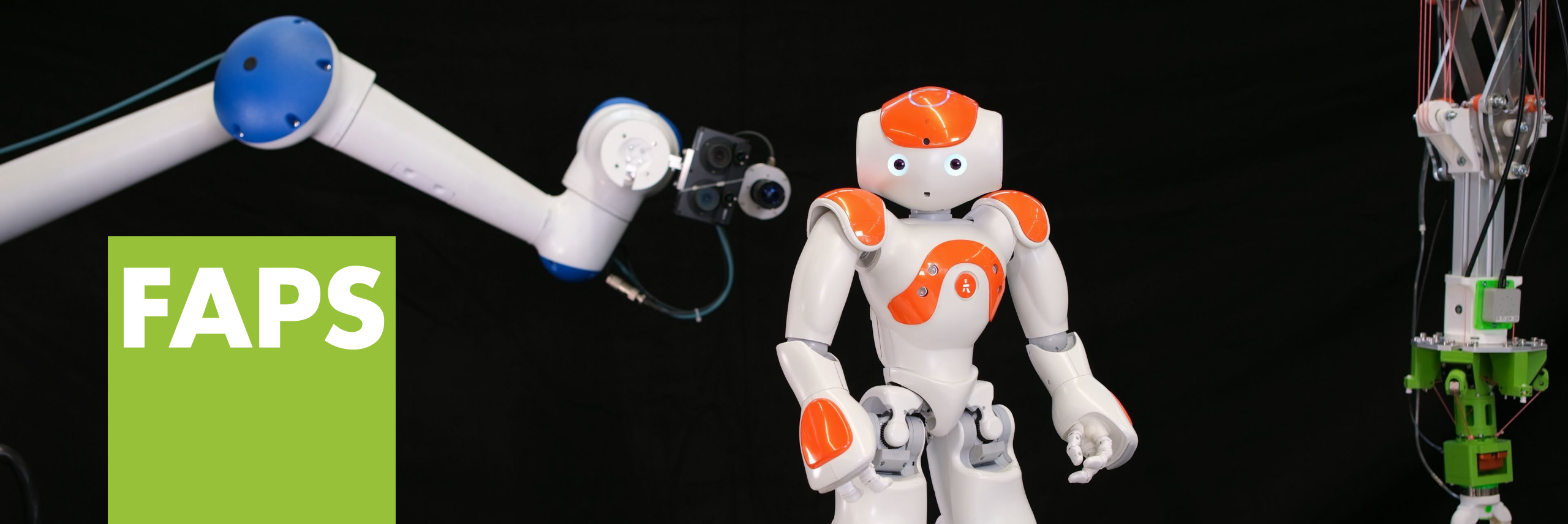
Glöckler, M.: „Simulation mechatronischer Systeme“, Springer, 2018; ISBN 978-3-658-20702-1

Siciliano, B., Khatib, O.: „Handbook of Robotics“, Springer, 2008; ISBN 978-3-540-30301-5

VDI 3633 „Simulation von Logistik-, Materialfluss-und Produktionssystemen“

Rabe M., Spieckermann S., Wenzel S.: Verifikation und Validierung für die Simulation in Produktion und Logistik





Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



Friedrich-Alexander-Universität
Technische Fakultät

DANKE